KSPM Documentation

1 | Kubernetes security

The Kubernetes Security Posture Management (KSPM) capability, driven by the KSPM Connector component, is positioned as a core security module within Cortex Cloud. KSPM is a lightweight cloud-native solution for Kubernetes security, both posture management and real-time protection that automatically discovers assets, enforces policies, and scans for vulnerabilities, malware, secrets, and misconfigurations across the environment.

The Cortex Cloud KSPM offering focuses on comprehensive security posture and compliance checks:

- Inventory and visibility: KSPM provides full visibility into your Kubernetes cluster and resources, including namespaces, nodes, and workloads. The KSPM dashboard provides a visual overview, including inventory insights and cluster protection coverage to show which clusters have no protection solution deployed.
- Compliance and misconfiguration detection: KSPM leverages hundreds of out-of-the-box KSPM rules. It detects compliance violations and misconfigurations using built-in and custom rules mapped to compliance controls. Compliance checks include CIS Benchmarks for both managed and unmanaged Kubernetes distributions. Custom rules are supported using Rego for the KSPM Connector and Python for Cortex XDR agent endpoints.
- Vulnerability, malware, and secret scanning: KSPM receives critical security information related to vulnerabilities (via AVA scan), malware, secrets, and other available scanners. AVA scan is a vulnerability identification and prioritization engine that discovers security vulnerabilities in Kubernetes cluster nodes and correlates findings with running workloads. Malware scanning detects suspicious or malicious code patterns within cluster workloads. Secret scanning identifies exposed credentials, API keys, authentication tokens, and other sensitive data that could be exploited for unauthorized access. The KSPM dashboard includes metrics for malware detected and secrets detected in clusters, enabling security teams to prioritize remediation efforts.
- Policy enforcement and control: An admission controller ensures all new resources meet required security and governance standards, enhancing the overall security posture of your environment. The admission controller intercepts requests to the Kubernetes API server before they are persisted, allowing enforcement of access control, image assurance, and security configurations.
- Centralized policy management: The offering provides centralized Cloud Workload Policy management and enforcement at runtime when the admission controller is used.
- Real-time threat detection and response (XDR): Beyond static posture, the solution provides active XDR capabilities to detect and intercept threats as they happen. It monitors live cluster activity to identify malicious behavior, zero-day attacks, and unauthorized runtime changes. This ensures that instead of just finding vulnerabilities, you are actively defending your environment against ongoing attacks with immediate visibility and automated response.

2 | KSPM limitations and system components

Refer to KSPM components and limitations for essential details on system architecture, resource requirements, and operational constraints when setting up your environment.

> **NOTE:**
>
> The CPU and memory limits can be adjusted to align with your resource needs. For configuration steps, refer to Configure Kubernetes Connector resource limits.

KSPM system components

Core components (continuously running):

Component Name	Description	Type	Priority Class**	Replicas	CPU (Req/Limit)	Memory (Req/Limit)
cortex-manager	This is the system's central engine that continuously monitors and aligns the cluster. It ensures the actual state of your deployment always matches your requested settings by automatically correcting any discrepancies.	Deployment	cortex-critical	1	50m / 100m	100Mi / 200Mi
cortex-admission	The cortex-admission acts as the system validator. It reviews every request to ensure it meets safety and configuration standards before allowing it to proceed.	Deployment	cortex-critical	3	50m / 100m	150Mi / 200Mi
cortex-monitoring	The cortex-monitoring component provides visibility into the health and performance of the system. It tracks metrics, logs, and health checks, acting as an early warning system that alerts operators of bottlenecks or failures.	Deployment	cortex-high	1	30m / 100m	100Mi / 200Mi

Scheduled components:

Component Name	Description	Type	Priority Class	Schedule/Trigger	CPU (Req/Limit)	Memory (Req/Limit)
cortex-syncer	The cortex-syncer ensures that information stays consistent across different environments (like the cloud and your local cluster) by constantly updating and matching records.	CronJob	cortex-high	Every hour	50m / 150m	50Mi / 200 Mi
cortex-job-orchestrator	It handles the scheduling and sequencing of background jobs, ensuring they run in the right order and at the right time without overlapping.	CronJob	cortex-high	Daily (configurable)	25m / 50m	75Mi / 100Mi
cortex-node-vm-discovery	It automatically scans the environment to identify and catalog all Virtual Machines (VMs) and nodes that need to be managed or protected.	Job*	cortex-high	Ephemeral	15m / 50m	50Mi / 100Mi
cortex-inventory-collector	It gathers a detailed inventory or list of all hardware and software components currently running in the cluster so you always know what you have.	Job*	cortex-high	Ephemeral	25m / 50m	50Mi / 100Mi

Component Name	Description	Type	Priority Class	Schedule/Trigger	CPU (Req/Limit)	Memory (Req/Limit)
cortex-compliance-node-scanner	It checks individual nodes against industry security standards to ensure the underlying servers are configured safely and legally.	Job*	cortex-high	Ephemeral	25m / 100m	50Mi / 100Mi
cortex-compliance-k8s-scanner	Specifically looks at your Kubernetes settings (like namespaces and RBAC) to ensure your cluster configuration meets strict security policies.	Job*	cortex-high	Ephemeral	200m / 50m	50Mi / 100Mi
cortex-host-scanner	It performs deep scans of the host operating system to find vulnerabilities, misconfigurations, or hidden security threats.	Job*	cortex-high	Ephemeral	300m / 1000m	50Mi / 13Gi
cortex-scan-result-publisher	It collects the raw data from all the other scanners, cleans it up, and sends it to your dashboard so you can actually read the results.	Job*	cortex-high	Ephemeral	30m / 50m	50Mi / 500Mi

*Spawned as ephemeral processes by the job-orchestrator.

Optional components:

Component Name	Description	Type	Priority Class	Scaling Behavior	CPU (Req/Limit)	Memory (Req/Limit)
cortex-realtime	The cortex-realtime is the system's high-speed streaming engine. It provides instantaneous data processing and analysis, allowing the cluster to react to security events or configuration changes the moment they occur. Refer to the Realtime Protection section in Onboard the Kubernetes Connector	DaemonSet	cortex-critical	1 pod per node	200m / 1500m	600Mi / 2Gi

** A PriorityClass is a non-namespaced Kubernetes resource that defines an importance level for pods. This influences their scheduling order and determines if they can evict (preempt) lower-priority pods during resource scarcity to ensure security continuity.

The following PriorityClasses are defined for the Kubernetes Connector:

- cortex-critical (Value: 1,000,010,000): Highest priority for mission-critical security components, including cortex-manager, cortex-admission-controller, and cortex-realtime.
- cortex-high (Value: 100,000,000): High priority for all other KSPM workloads, ensuring they are prioritized over standard, non-essential application traffic.

KSPM limitations

Supported operating systems: Linux

> **NOTE:**
>
> Windows and Unix are not supported.

Supported CPU architecture:

- AMD64 (x86_64)
- ARM64 (aarch64)

Technical specifications

- CPU units: 1000m = 1 CPU core
- Memory units: Mi = Mebibytes, Gi = Gibibytes

2.1 | Configure Kubernetes Connector resource limits

The Kubernetes Connector consists of multiple components that require CPU and memory allocation. Follow the steps to set the limits that align with your resource needs.

1. From the Kubernetes Connect wizard, click Generate to download the `K8s-security-profile-panw-DATE.values.yaml` file from the first step.

2. Edit the file to adjust the requests and limits of the workloads.

   Use this example as a reference when configuring your file:

```
konnector-template:
  enabled: true

  # ================================================================================
  # Core — Cortex Manager (k8sConnector) — full override
  # ================================================================================
  coreValues:
    resources:
      k8sConnector:
        overrides:
          requests:
            cpu: "300m"
            memory: "256Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"

  # ================================================================================
  # Monitoring — OpenTelemetry Collector (otel) — full override
  # ================================================================================
  monitoringValues:
    enabled: true
    resources:
      otel:
        overrides:
          requests:
            cpu: "100m"
            memory: "200Mi"
          limits:
            cpu: "300m"
            memory: "400Mi"

  # ================================================================================
  # Posture — admission controller, orchestrator, and spawned jobs
  # ================================================================================
  postureValues:
    enabled: true
    admissionController:
      enabled: true
      failurePolicy: Ignore
    jobOrchestrator:
      scanIntervalHours: 12

    resources:
      # Admission Controller — full override
      admissionController:
        overrides:
          requests:
            cpu: "100m"
            memory: "300Mi"
          limits:
            cpu: "300m"
            memory: "600Mi"

      # Job Orchestrator — CPU request + CPU limit only
      # (memory keeps defaults: requests.memory=75Mi, limits.memory=100Mi)
      jobOrchestrator:
        overrides:
          requests:
            cpu: "50m"      # was 25m
          limits:
            cpu: "100m"     # was 50m

      # Resources for jobs spawned by the orchestrator
      jobResources:
        # Host Scanner — memory limit only
        # (CPU and memory request keep defaults)
        host-scanner:
          overrides:
            limits:
```

```
            memory: "8Gi"   # was 4.5Gi

        # Inventory Collector — CPU request + CPU limit only
        # (memory keeps defaults: requests.memory=50Mi, limits.memory=100Mi)
        inventory-collector:
          overrides:
            requests:
              cpu: "50m"     # was 25m
            limits:
              cpu: "100m"    # was 50m

        # Compliance K8s Scanner — full override
        compliance-k8s-scanner:
          overrides:
            requests:
              cpu: "300m"
              memory: "100Mi"
            limits:
              cpu: "750m"
              memory: "200Mi"

        # Scan Result Publisher — full override
        scan-result-publisher:
          overrides:
            requests:
              cpu: "50m"
              memory: "100Mi"
            limits:
              cpu: "100m"
              memory: "750Mi"

        # Node VM Discovery — full override
        node-vm-discovery:
          overrides:
            requests:
              cpu: "30m"
              memory: "100Mi"
            limits:
              cpu: "100m"
              memory: "200Mi"

    # ============================================================================
    # Realtime Protection — DaemonSet agent — full override (incl. ephemeral storage)
    # ============================================================================
    realtimeProtectionValues:
      enabled: true
      distribution:
        id: distID
        url: https://distributions-dev.traps.paloaltonetworks.com
      platform:
        bottlerocket: false
        gcos: false
      resources:
        agent:
          overrides:
            requests:
              cpu: "300m"
              memory: "800Mi"
              ephemeral-storage: "6Gi"
            limits:
              memory: "3Gi"
              ephemeral-storage: "12Gi"
```

3. Deploy the Kubernetes Connector using Helm:

```
helm upgrade --install konnector oci://us-central1-docker.pkg.dev/qa2-test-YYY/agent-docker/helm/konnector-launcher \
--wait-for-jobs \
--create-namespace \
--namespace panw \
--values K8s-security-profile-panw-DATE.values.yaml \
--set-file konnector-upgrader.rawValuesContent=K8s-security-profile-panw-DATE.values.yaml
```

3 | Supported Kubernetes distributions

The following are the supported Kubernetes platform versions for the Kubernetes connector (Posture Management). The table shows the latest version that is supported. We support n-3 versions of each supported Kubernetes environment.

Kubernetes Environment	Notes
Managed clusters	• Amazon Elastic Kubernetes Service (EKS) • Microsoft Azure Kubernetes Service (AKS) • Google Kubernetes Engine (GKE) **NOTE:** Does not include Autopilot.
Managed OpenShift	Managed Openshift clusters, including ROSA (Red Hat OpenShift on AWS), are supported.
Self-Managed	We support every CNCF-certified Kubernetes solution. We've tested our solution on: • Self-managed vanilla/on-premise Kubernetes clusters. • Self-managed OpenShift Kubernetes clusters. • Rancher Distributions (RKE and RKE2).

The following are the Kubernetes platforms that are supported with Cortex XDR agents (Real-time protection).

This table shows the Kubernetes platform versions that have been compatibility tested. The table shows the latest version that has been tested. All versions that are not EOL, up to the latest version are supported.

Linux Kubernetes Platform	Version
Unmanaged Kubernetes (k8s)	1.30
Amazon Elastic Kubernetes Service (EKS)	1.33
BottleRocket OS x86_64 User mode agent only	
BottleRocket OS aarch64 User mode agent only	
Microsoft Azure Kubernetes Service (AKS)	1.33
CBL-mariner 2 x86_64	
Google Kubernetes Engine (GKE)	1.33

Linux Kubernetes Platform	Version
Google Container-Optimized OS (COS)* x86_64 User mode agent only	
Google Kubernetes Engine (GKE) Autopilot	
Oracle Kubernetes Engine (OKE)	1.33
Red Hat Openshift Container Platform (OCP)	4.16
RHCOS* x86_64 User mode agent only	
SUSE Rancher Kubernetes Engine 2 (RKE2)	1.28
Talos	1.8.3

NOTE:

In Google Container-Optimized OS release 100 and earlier, where the FANOTIFY EXEC flag is not supported, the Kernel configuration may be partial for the user mode agent to properly function. In such cases, the agent will fallback to asynchronous mode.

In RHCOS version 4.12 and earlier, the Kernel configuration may be partial for the user mode agent to properly function. In such cases, the agent will fallback to asynchronous mode.

4 | Onboard the Kubernetes Connector

Abstract

To onboard your Kubernetes cluster, choose the capabilities that fit your needs and download the Helm chart values. Install the Helm charts in your Kubernetes environment to grant Cortex Cloud permissions to collect the data.

Follow this wizard to deploy your Kubernetes Connector. The Kubernetes onboarding wizard is designed to facilitate the seamless setup of Kubernetes data into Cortex Cloud. The guided experience requires minimal user input; simply select the capabilities that fit your needs and download the custom installer file. For full control of the setup, you can use the advanced settings. Based on the onboarding settings, Cortex Cloud then creates a custom installer file for running in your Kubernetes environment. This file, once executed in your Kubernetes environment, grants Cortex Cloud the necessary permissions to collect the data. The installer file must be executed in your Kubernetes environment to complete the onboarding process. The connector then appears in Kubernetes Connectors.

1. Navigate to Settings → Data Sources & Integrations.
2. On the Add Data Sources & Integrations page, click Create Integration, search for Kubernetes, then hover over it and click Add Another Instance.
3. In the Kubernetes Connect onboarding wizard, enable the solutions that fit your needs:

- Posture Management: (Enabled by default) A lightweight posture management solution for continuous discovery, policy enforcement, and proactive scanning of vulnerabilities, secrets, malware, compliance, and misconfigurations.
- Realtime Protection: A solution that monitors workloads in real time to detect and block malicious activity, instantly preventing attacks as they happen.

4. (Optional) Click Edit to configure advanced settings and then click Apply Changes:

- Posture Management:

Setting	Notes
Scan Cadence (Hours)	Define how often to scan (from every one to 24 hours). Default is 12 hours.
Policy Enforcement by the Admission Controller	Select to allow enforcement policies to be configured, ensuring that only compliant resources are admitted into the cluster.
Registry Scanning (OpenShift Only)	Select this option to scan OpenShift Platform Registry images for vulnerabilities, malware, and exposed secrets. Select the scanning configuration option to enable security checks for your images: ◦ All (Default) Scans all container images, including all versions (tags), in all discovered repositories. ◦ Latest tag: Scans only images tagged 'latest' in all discovered repositories. ◦ Day modified: Scans container images created or modified in the last few days. You can select a range of up to 90 days for the scan. The default is set to 7. Refer to OpenShift container registry for information on the instances that were automatically created by the Kubernetes deployment.

- Realtime Protection:

> **NOTE:**
>
> This option is not supported for Fargate.

> **NOTE:**
>
> Enabling Realtime Protection installs the agent on your Kubernetes clusters as a DaemonSet.

Setting	Notes
Node Selector	Enter node labels to have the agent run on nodes that match the node labels. Ensure that you use the `key:value` format. > **NOTE:** > > Duplicate keys are not allowed.
Run on all nodes (Including Master)/Run only on master node	Select one of the options.

Setting	Notes
Endpoint tags	Select endpoint tags to assign to agents during installation. You can refer to the full list of tags under All endpoints after the agent installation is complete.
Deployment Platform	Select the Kubernetes deployment platform: ◦ Standard ◦ Bottlerocket OS ◦ Google GCOS ◦ OpenShift

-

5. (Optional) Click Edit Profile to customize the Kubernetes Connector's profile:

Setting	Notes
Profile Name	A profile name is automatically generated, including the date and time of creation. You can manually change the profile name.
Version	Lists the most up-to-date KSPM connector bundle versions available: • 2.0.*- Current series with private registry support and on-demand scanning • 1.4.* - Legacy series
Cluster Resource Identifier	(Optional) Enter the Kubernetes cluster resource identifier. If you do not specify the resource identifier, the installer will identify the cluster on its own. **NOTE:** For Fargate, you must provide the cluster resource identifier. The format of the identifier is `arn:aws:eks:<region>:<account-id>:cluster/<cluster-name>`.

Setting	Notes
Namespace	Enter the name for the Kubernetes namespace. The default is "panw". To ensure proper data parsing in an AWS Fargate environment, a Fargate Profile must be explicitly configured for the namespace where the connector is installed (typically panw) and for the kube-system namespace if the cluster is fully Fargate-based. Because the system identifies Fargate clusters by scanning for active workloads during deployment, a Fargate profile that contains no running pods will not be recognized as such. Furthermore, since this detection occurs at installation, any transition from EC2 to Fargate requires an agent update to trigger a new scan and ensure the environment is correctly identified and monitored.
Proxy Gateway	Enable this option if network traffic between Cortex Cloud and your Kubernetes cluster must route through a proxy gateway. Enter the following details: • Proxy IP: The full IP address and port number for your HTTP proxy server. For example: `192.168.1.1:8080` • Authentication: Select None or Basic. Enter the username and password for a proxy user account that has permission to pass traffic to the Kubernetes cluster. **NOTE:** Basic authentication is only supported in Posture Management. If deploying Realtime Protection, select None .

Setting	Notes
Auto Upgrade	Enable Auto Upgrade to ensure the Kubernetes Connector and its installed capabilities are automatically updated to a newer version when available. This minimizes manual maintenance and ensures continuous access to the latest features and security patches. Select the Upgrade Strategy: • Latest Available Version (GA): Automatically upgrade to the newest version as soon as it is released to gain immediate access to all new features. • One release before the latest one (N-1): Maintain a policy to always remain one version behind the latest available release. **NOTE:** If you install the latest version but select the N-1 strategy, this policy will take effect starting from the next upgrade cycle (it will not immediately downgrade your current installation). If you choose an older version and keep the latest strategy, the latest version will be installed. Select Advanced to customize the upgrade schedule. Define whether to be upgraded immediately or to delay the upgrade by a specified number of days. You can then specify the preferred day and time for the upgrade to be applied.

6. Click Generate.
7. To complete the onboarding of the Kubernetes Connector, you must download the Helm chart values `values.yaml` and run it in your Kubernetes environment: `helm repo add cortex https://paloaltonetworks.github.io/cortex-cloud --force-update`
8. Install the Helm charts in your Kubernetes environment: `helm upgrade --install konnector cortex/konnector --wait-for-jobs --create-namespace --namespace panw --values <profile-name>.values.yaml`
9. Verify the deployment succeeded when you see Status: Deployed.

   When the Kubernetes Connector is deployed, the initial discovery scan is started, and the connector appears in Data Sources & Integrations → Kubernetes → Kubernetes Connectors.

4.1 | OpenShift container registry

The OpenShift Container Registry provides visibility into the container images stored within your OpenShift environment. When you deploy the Kubernetes Connector on an OpenShift platform, the system automatically discovers and creates registry instances for monitoring.

When adding an instance of Kubernetes from Data Sources & Integrations, the setting Registry Scanning (OpenShift Only) must be configured to enable automatic scanning of an OpenShift container registry. By default, scanning operates in a concurrency-limited mode (10 images at a time) to minimize resource impact on the cluster while maintaining thorough security coverage.

Manage OpenShift registry instances

You can view and manage the registry instances automatically created by your OpenShift Kubernetes deployment.

- Data source page: Displays a list of all discovered OpenShift registry instances.
- Key details: For each instance, you can quickly see the status, instance name, and associated cluster name.

View instance details

To gain deeper insights into a specific registry, click on one of the instances from the table of the OpenShift Container Registry. This opens a page that includes detailed information about the particular instance.

The detailed dashboard provides the following information:

- Status: The current operational state of the registry instance.
- Kubernetes Connector Status: Indicates the health and connectivity of the connector managing the registry.
- Repositories: The total count of image repositories discovered within the registry.
- Scan Mode: Displays the current scanning configuration.
- Registry Scanning: Shows the real-time status of the cluster. Click on the Status, which navigates you to a dashboard that shows the total number of assets and their current status, and the Health Audits of the specific cluster.

Repository inventory

The instance details page includes a granular list of all repositories found within the OpenShift registry. This list includes:

- Name: The identifier for the specific image repository.
- Repository Type: Categorization of the repository within the OpenShift environment.

Actions

From the top left side of the page, click the More Options (⋮) to:

- Exclude/Include images: You can configure scoping rules to include or exclude specific images and repositories from scans based on their names or tags. While updates apply to all future scans, previously excluded images will only be processed during a subsequent system-wide rescan.
- Discover Now: Triggers the discovery and then scanning of newly discovered assets.

Access OpenShift from Kubernetes

You can also view important details about your OpenShift registry from the Kubernetes data source. From the list of clusters in the Kubernetes table, select the OpenShift registry to view:

- Connector Details: Shows last scan, connector status and connector version of the OpenShift platform.
- Cluster Details: Click the link to view the asset card of the OpenShift cluster.
- Deployment Details: Shows deployment method used and the deployment date.

4.2 | Deploy the Kubernetes Connector via GitOps

Deploy and manage the Kubernetes Connector (konnector-bundle Helm chart) across multiple Kubernetes clusters using a complete GitOps setup. Implement this repository structure to use either ArgoCD or Flux CD as your primary controller.

GitOps controller comparison

- ArgoCD: Recommended if you want a visual UI for managing deployments, need real-time drift detection, require built-in rollback with history, and prefer a GUI-based workflow.
- Flux CD: Recommended if you want native Helm release management (`helm install`), need Helm hooks support, and prefer a pure CLI/GitOps workflow.

Prerequisites

Before you begin, ensure you have the following:

- A Kubernetes cluster with `kubectl` configured.
- The `helm` CLI installed for pulling charts from OCI registries
- The `flux` CLI and/or `argocd` CLI installed.
- A Git repository accessible from within the cluster.
- Access to the `konnector-bundle` OCI registry.

4.2.1 | Repository architecture and tenant management

The GitOps repository uses self-contained tenant folders, allowing each tenant to have its own chart, values, and overlays.

Directory structure

- `charts/`: Contains Helm charts organized by tenant.
  - `konnector-bundle/`: The Helm chart pulled from the Cortex tenant OCI registry.
  - `tenant-values.yaml`: Your specific tenant value overrides downloaded from Cortex.
  - `overlays/`: Contains the Kustomize config and specific application patches for `flux/` and `argocd/`.

Prepare the deployment files in the repo

Regardless of your chosen Controller, you must first prepare the tenant directory:

1. Add a new Kubernetes integration in Cortex, from Settings → Data Sources & Integrations → Add New → Kubernetes.
2. Click Add and configure the Kubernetes Connect settings, and then click Generate.

   > **NOTE:**
   >
   > Make sure to disable auto upgrade, so it does not conflict with Flux or ArgoCD state management.

3. Follow the steps under How do I install?
   1. In step 3, instead of deploying directly, pull the charts to add them to your GitOps repository.
      - Copy the Helm release repo path and the version number.

        ```
        helm pull <repo-path> --version <version> --untar
        ```

   2. Copy the Connector bundle directory directly to the charts directory.
   3. Copy the values file directly to the charts directory.
4. Create an overlays directory in the charts directory.

- Flux

    1. Create a `helm-release.yaml` file that contains a Flux HelmRelease custom resource definition (CRD) object. This object specifies the base chart location in the repository and a ConfigMap generated from Kustomization instructions. These instructions point to tenant-specific values that patch the default values. Include the base Flux instructions for maintaining this release in the cluster.

```
# Flux HelmRelease for KSPM Agent
#
# This is a self-contained HelmRelease that deploys the konnector-bundle chart
# from the Git repository. No separate base or patch files are needed.
#
# Prerequisites:
#   - Flux is installed on the cluster
#   - A GitRepository source named "kspm-gitops" exists in flux-system namespace
#   - The tenant's konnector-bundle chart is at the path specified below
#
# Value merge order (last wins):
#   1. Chart's built-in values.yaml (always loaded first by Helm)
#   2. valuesFrom entries (ConfigMaps/Secrets, processed in order)
#   3. spec.values (inline values, highest priority)
#
# NOTE: If you change the namespace from "panw" to something else, you must
# update it in ALL of these places:
#   1. metadata.namespace below
#   2. tenant-values.yaml (global.namespace)
#   3. configMapGenerator namespace in kustomization.yaml
# The namespace is automatically created by Helm via install.createNamespace.
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: kspm-agent
  # Target namespace — change here if using a custom namespace.
  # Helm will create this namespace automatically (install.createNamespace: true).
  namespace: panw
  labels:
    app.kubernetes.io/part-of: kspm-agent
    app.kubernetes.io/managed-by: flux
spec:
  interval: 5m
  timeout: 5m
  chart:
    spec:
      # Path to the tenant's chart (relative to Git repo root)
      # Replace <TENANT_ID> with your actual tenant ID
      chart: ./charts/konnector-bundle
      sourceRef:
        kind: GitRepository
        name: kspm-gitops
        namespace: flux-system
      reconcileStrategy: Revision
  install:
    createNamespace: true
    remediation:
      retries: 3
  upgrade:
    remediation:
      retries: 3
    cleanupOnFail: true
  uninstall:
    keepHistory: false
  # Values sources — processed in order, later entries override earlier ones
  valuesFrom:
    # Tenant-specific overrides loaded from a ConfigMap
    # (equivalent to: helm install -f tenant-values.yaml)
    - kind: ConfigMap
      name: tenant-values
      valuesKey: values.yaml
  values:
    # Inline value overrides (highest priority)
    # Uncomment and set values here to override everything above
    # global:
    #   optionalValues:
    #     CONSOLE_LOG_LEVEL: "DEBUG"
```

    > **NOTE:**
    >
    > If the namespace provided in the tenant is different from panw, change the namespace in both the Kustomization and HelmRelease files.

- ArgoCD

1. Create an `application.yaml` file in the overlays directory.

```
# ArgoCD Application for KSPM Agent
#
# This is a self-contained ArgoCD Application that deploys the konnector-bundle
# chart from the Git repository. No separate base or patch files are needed.
#
# Prerequisites:
#   - ArgoCD is installed on the cluster
#   - The Git repository is accessible from ArgoCD
#   - The tenant's konnector-bundle chart is at the path specified below
#
# Value merge order (last wins):
#   1. Chart's built-in values.yaml (always loaded first by Helm)
#   2. valueFiles entries (processed in order)
#   3. Inline values (highest priority)
#
# NOTE: If you change the namespace from "panw" to something else, you must
# update it in ALL of these places:
#   1. spec.destination.namespace below
#   2. AppProject destinations (if using a custom project)
#   3. tenant-values.yaml (global.namespace)
# The namespace is automatically created via syncOptions CreateNamespace=true.
#
# Replace the following placeholders before deploying:
#   <REPO_URL>   - Your Git repository URL (e.g., https://gitlab.example.com/kspm/kspm-gitops.git)
#   <TENANT_ID>  - Your tenant ID
---
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: kspm
  namespace: argocd
  labels:
    app.kubernetes.io/part-of: kspm-agent
    app.kubernetes.io/managed-by: argocd
spec:
  description: "KSPM Agent GitOps project"
  destinations:
    # Target namespace - change here if using a custom namespace
    - namespace: panw
      server: https://kubernetes.default.svc
  sourceRepos:
    - "*"
  # Allow all cluster-scoped resources that the agent might need
  clusterResourceWhitelist:
    - group: ""
      kind: Namespace
    - group: rbac.authorization.k8s.io
      kind: ClusterRole
    - group: rbac.authorization.k8s.io
      kind: ClusterRoleBinding
    - group: policy
      kind: PodSecurityPolicy
  # Allow all namespaced resources
  namespaceResourceWhitelist:
    - group: "*"
      kind: "*"
---
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: kspm-agent
  namespace: argocd
  labels:
    app.kubernetes.io/part-of: kspm-agent
    app.kubernetes.io/managed-by: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: kspm
  # Use 'sources' (plural) to enable $values references from the same repo
  sources:
    # Source 1: The Helm chart
    - repoURL: <REPO_URL>
      path: charts/konnector-bundle
      targetRevision: main
      helm:
        releaseName: kspm-agent
        valueFiles:
          # Chart's own values.yaml
          - values.yaml
          # Tenant-specific overrides - path relative to repo root via $values
          - $values/charts/tenant-values.yaml
```

```
      # Inline values (highest priority)
      # values: |
      #   global:
      #     optionalValues:
      #       CONSOLE_LOG_LEVEL: "DEBUG"
  # Source 2: Values reference (same repo, enables $values/ prefix in valueFiles)
  - repoURL: <REPO_URL>
    targetRevision: main
    ref: values
destination:
  server: https://kubernetes.default.svc
  # Target namespace — change here if using a custom namespace
  namespace: panw
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    # Automatically creates the namespace if it doesn't exist
    - CreateNamespace=true
    - ServerSideApply=true
  retry:
    limit: 3
    backoff:
      duration: 5s
      factor: 2
      maxDuration: 3m
# Ignore differences in resources that are modified at runtime by the konnector
# manager. Without this, ArgoCD's selfHeal would revert runtime changes.
ignoreDifferences:
  # The backend-auth-secret tokens are placeholder values in the chart template.
  # The cortex-manager replaces them with real tokens at runtime.
  - group: ""
    kind: Secret
    name: backend-auth-secret
    jsonPointers:
      - /data
  # The admission-control-tls secret is created by the admission controller at runtime
  - group: ""
    kind: Secret
    name: admission-control-tls
    jsonPointers:
      - /data
  # The webhook CA bundle is injected by the admission controller
  - group: admissionregistration.k8s.io
    kind: ValidatingWebhookConfiguration
    jsonPointers:
      - /webhooks/0/clientConfig/caBundle
```

2. Create a `kustomization.yaml` file in the overlays directory.

```
# ArgoCD Kustomization for KSPM Agent
#
# This kustomization deploys the ArgoCD AppProject and Application.
#
# To deploy:
#   kubectl apply -k charts/tenants/<TENANT_ID>/overlays/argocd/
#
# Directory structure expected:
#   charts/tenants/<TENANT_ID>/
#   ├── tenant-values.yaml          # Your tenant-specific Helm values
#   ├── konnector-bundle/           # The Helm chart
#   └── overlays/argocd/
#       ├── kustomization.yaml      # This file
#       └── application.yaml        # The AppProject + Application
---
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - application.yaml
```

5. Add the new tenant directory to Git, commit the changes, and then push them to your main branch.

4.2.2 | Deploy with Flux CD

Deploy the Cortex KSPM agent to on-premises Kubernetes clusters using Flux CD. Flux CD is a declarative GitOps continuous delivery tool that automates agent deployment and configuration synchronization from a Git repository.

1. Install the following Flux controllers:

- Source Controller
- Kustomize Controller
- Helm Controller
- Notification Controller

```
flux install
```

2. Verify that all controllers are running.

```
flux check

#Expected output:
✔ helm-controller: deployment ready
✔ kustomize-controller: deployment ready
✔ notification-controller: deployment ready
✔ source-controller: deployment ready
```

3. Create the authentication secret.

```
# For username/password or token (HTTP)
kubectl create secret generic gitlab-auth \
  --namespace=flux-system \
  --from-literal=username=<GIT_USERNAME> \
  --from-literal=password=<GIT_PASSWORD_OR_TOKEN>

#For SSH-based authentication
flux create secret git gitlab-auth \
  --namespace=flux-system \
  --url=ssh://git@gitlab.example.com/kspm/kspm-gitops.git \
  --private-key-file=<path-to-private-key>
```

4. Create a GitRepository source.

```
flux create source git kspm-gitops \
  --url=https://gitlab.example.com/kspm/kspm-gitops.git \
  --branch=main \
  --interval=1m \
  --secret-ref=gitlab-auth \
  --namespace=flux-system
```

5. Verify the source is connected.

```
flux get source git kspm-gitops

#Expected output:
NAME         REVISION          SUSPENDED  READY  MESSAGE
kspm-gitops  main@sha1:abc123  False      True   stored artifact for revision 'main@sha1:abc123'
```

6. Deploy the connector.

```
```bash
flux create kustomization kspm-agent \
--source=GitRepository/kspm-gitops \
--path="./charts/overlays" \
--prune=true \
--interval=5m
```
```

4.2.3 | Deploy with ArgoCD

Deploy the Cortex KSPM agent to on-premises Kubernetes clusters using ArgoCD. ArgoCD is a declarative GitOps continuous delivery tool that automates agent deployment and configuration synchronization from a Git repository.

1. Install ArgoCD.

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

2. Connect ArgoCD to a git repository.

```
# HTTPS based authentication
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
```

```
  name: gitlab-repo
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
stringData:
  type: git
  url: https://gitlab.example.com/kspm/kspm-gitops.git
  username: <GIT_USERNAME>
  password: <GIT_TOKEN>
EOF

# SSH based authentication
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: gitlab-repo
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: repository
stringData:
  type: git
  url: git@gitlab.example.com:kspm/kspm-gitops.git
  sshPrivateKey: |
    -----BEGIN OPENSSH PRIVATE KEY-----
    <your-private-key>
    -----END OPENSSH PRIVATE KEY-----
EOF
```

3. Apply the ArgoCD Application manifest.

```
kubectl apply -f charts/overlays/application.yaml
```

4.2.4 | Private registry support

Private registry support enables the Cortex KSPM Connector to pull container images from private Docker registries using Kubernetes image pull secrets. This capability eliminates the requirement to push connector images to public registries. Maintain complete control over image distribution and enforce internal security policies for container image management.

Private registry support provides image pull authentication for the KSPM Connector workloads. The feature does not manage registry credentials, configure registry endpoints, or validate image signatures. Registry credential management remains your responsibility.

Roles and responsibilities

Private Registry Support facilitates a delegation model for your infrastructure and security tasks:

- Kubernetes Administrators (Infrastructure): Create and maintain Kubernetes image pull secrets (konnector-docker-secret) containing private registry credentials. Verify that the connector namespace has access to the required secrets.
- Security Teams (Operations): Deploy the KSPM Connector using private registry image references. Validate that container images are successfully pulled, and connector workloads are operational.

Benefits

Private Registry Support transitions you from public-only image distribution to a controlled, private image delivery model:

- Eliminating public registry dependency: Removing the requirement to publish connector images to public registries, reducing external exposure, and simplifying compliance with internal image distribution policies.
- Enforcing image control: Centralizing container image management in private registries lets you apply custom scanning, signing, and access control policies before images reach the cluster.
- Reducing supply chain risk: Maintaining complete control over image provenance and distribution prevents unauthorized image modifications or substitutions.

Prerequisite

Before you begin, make sure you have the following:

- A private Docker registry accessible from the Kubernetes cluster (for example, Docker Hub private repositories, Amazon ECR, Azure Container Registry, Google Artifact Registry, or self-hosted registries).
- Kubernetes image pull secret (`konnector-docker-secret`) created in the connector namespace with valid registry credentials.
- Container images for the KSPM Connector are available in the private registry.

How to deploy KSPM Connector with private registry images

1. Create Kubernetes image pull secret

   Create a Kubernetes image pull secret in the connector namespace with credentials for the private registry:

   ```
   kubectl create secret docker-registry konnector-docker-secret \
   --docker-server=<REGISTRY_URL> \
   --docker-username=<USERNAME> \
   --docker-password=<PASSWORD> \
   --docker-email=<EMAIL> \
   -n panw
   ```

   Replace the following placeholders:

   - `<REGISTRY_URL>`: The private registry endpoint (for example, `myregistry.azurecr.10`).
   - `<USERNAME>`: The registry username or service principal ID.
   - `<PASSWORD>`: The registry password or access token.
   - `<EMAIL>`: A valid email address (required by Kubernetes, but not used for authentication).

2. Configure Helm chart with private registry images

   Update the Helm chart `values.yaml` to reference images from the private registry:

   ```
   image:
     repository: <REGISTRY_URL>/cortex-kspm-connector
     tag: <IMAGE_TAG>
     pullPolicy: IfNotPresent
   imagePullSecrets:
     - name: konnector-docker-secret
   ```

   Replace the following placeholders:

   - `<REGISTRY_URL>`: The private registry endpoint.
   - `<IMAGE_TAG>`: The connector image tag (for example, `1.19.0`).

3. Deploy the KSPM Connector

   Deploy the KSPM Connector using the Helm chart with the updated configuration:

   ```
   helm install cortex-kspm-connector ./k8s-manager \
   -f values.yaml \
   -n panw \
   --create-namespace
   ```

4. Verify image pull success

   Verify that the connector pods successfully pulled images from the private registry:

   ```
   kubectl get pods -n panw -o wide
   kubectl describe pod <POD_NAME> -n panw
   ```

   Check the pod events for any `ImagePullBackOff` or authentication errors. If the image pull fails, verify that the `konnector-docker-secret` contains valid credentials and is accessible from the connector namespace.

How to update private registry credentials

If private registry credentials expire or change, update the image pull secret:

1. Delete the existing secret

   ```
   kubectl delete secret konnector-docker-secret -n panw
   ```

2. Create a new secret with updated credentials

```
kubectl create secret docker-registry konnector-docker-secret \
--docker-server=<REGISTRY_URL> \
--docker-username=<NEW_USERNAME> \
--docker-password=<NEW_PASSWORD> \
--docker-email=<EMAIL> \
-n panw
```

3. Restart connector pods

   Restart the connector pods to force an image re-pull with the new credentials:

   ```
   kubectl rollout restart deployment/cortex-kspm-connector -n panw
   ```

   Monitor pod restart completion:

   ```
   kubectl rollout status deployment/cortex-kspm-connector -n panw
   ```

Supported registry types

Private Registry Support is compatible with the following registry types:

- Docker Hub (private repositories)
- Amazon Elastic Container Registry (ECR)
- Azure Container Registry (ACR)
- Google Artifact Registry (GAR)
- JFrog Artifactory
- Harbor
- Self-hosted Docker Registry V2
- Any registry compatible with Docker Registry HTTP API V2

4.2.5 | GitOps lifecycle management

The core GitOps workflow requires that you make a change in Git, push it, and let your controller apply it.

Update tenant values and configurations

- Update values: Overwrite the `tenant-values.yaml` file with the newly downloaded file from Cortex, commit, and push.
- Update chart version: Delete the old `konnector-bundle` directory, run `helm pull` to download the new `<NEW_VERSION>`, commit, and push.
- Add inline overrides: Add specific configurations to your `helmrelease-patch.yaml` (Flux) or `application.yaml` (ArgoCD) . Inline values have the highest priority and will override both the chart defaults and `tenant-values.yaml`.

Namespace configuration

The target namespace is controlled by the `global.namespace` parameter in `tenant-values.yaml` (default: `panw`). If changing the namespace, you must update it in three places:

1. `global.namespace` in `tenant-values.yaml`.
2. The ConfigMap namespace in `overlays/.../kustomization.yaml`.
3. The target namespace in `helmrelease-patch.yaml` (Flux) or `spec.destination.namespace` in `application.yaml` (ArgoCD).

Day-to-day operations

- Suspend/disable auto-sync:
  - Flux: `Run flux suspend helmrelease kspm-agent -n panw` or set `suspend:true` in your HelmRelease spec via Git.
  - ArgoCD: Run `argocd app set kspm-agent --sync-policy none` or remove the `automated:` block from your Application spec via Git.
- Manual reconciliation/sync:
  - Flux: `flux reconcile kustomization kspm-agent<TENANT_ID>`.
  - ArgoCD: `argocd app sync kspm-agent`
- Uninstall a release:
  - Flux: Delete the Flux Kustomization (`flux delete kustomization kspm-agent-<TENANT_ID>`) to cascade and remove all managed resources.
  - ArgoCD: Delete the ArgoCD Application with the cascade flag (`argocd app delete kspm-agent --cascade`).

> **NOTE:**
>
> For Proof of Concept testing, the repository includes a `deploy-gitlab.sh` script to deploy a self-hosted GitLab EE server directly on your cluster.

4.2.6 | Troubleshooting GitOps deployments

Diagnose and resolve deployment or synchronization issues with your Kubernetes Connector. Use your controller's logs and diagnostic commands to investigate failures, force state reconciliations, and fix common errors for Flux and ArgoCD.

Controller logs

If resources fail to sync, check the controller logs:

- Flux logs: Git fetch issues: `kubectl logs -n flux-system deploy/source-controller`.
  - Helm install issues: `kubectl logs -n flux-system deploy/helm-controller`.
  - Build issues: `kubectl logs -n flux-system deploy/kustomize-controller`.
- ArgoCD logs:
  - Sync issues: `kubectl logs -n argocd deploy/argocd-application-controller`.
  - Git/Helm issues: `kubectl logs -n argocd deploy/argocd-repo-server`.

Advanced diagnostics

- View applied values (Flux): Run `helm get values kspm-agent -n panw -a` to see what values Flux successfully merged and applied.
- View rendered manifests (ArgoCD): Run `argocd app manifests kspm-agent` to see what ArgoCD would apply (dry-run) or `argocd app diff kspm-agent` to compare live vs. desired state.
- Force hard refresh (ArgoCD): If the repo is out of sync, force a re-fetch and re-render using `argocd app get kspm-agent --hard-refresh`.

Common issues diagnosis

Flux issues:

- GitRepository not ready: Check URL, credentials, and network connectivity.
- Kustomization failed: Check relative paths in your `kustomization.yaml` file.
- ConfigMap not found: Ensure the ConfigMap namespace identically matches the HelmRelease namespace.

ArgoCD issues:

- ComparisonError: Chart rendering failed. Check chart path and values.yaml syntax.
- SyncError: Resources cannot be applied to the cluster. Check RBAC, namespace constraints, or resource conflicts.
- OutOfSync but won't sync: Auto-sync has been disabled. Either manually sync or re-enable the auto-sync policy.

5 | KSPM dashboard

> **IMPORTANT:**
>
> Users can access all information on the dashboard when their user access is scoped to view All assets or assigned to the Instance Administrator role. Otherwise, users with granular scoping set to No assets or Select asset groups will have limited access to the dashboard. For more information on Scope-Based Access Control (SBAC), see Manage user scope in the Cortex Cloud Runtime Security Documentation or Manage user scope in the Cortex Cloud Posture Management Documentation.

The KSPM dashboard provides a centralized overview of your entire Kubernetes environment, enabling you to quickly identify risks and prioritize remediation actions. The KSPM dashboard is included in the predefined dashboards.

To access the KSPM dashboard, select Dashboards & Reports → Dashboard or Modules → Kubernetes Security → KSPM Dashboard . From the dashboard header, a drop-down menu lists all available predefined and custom dashboards. Select KSPM. You can use the filtering options at the top of the dashboard to focus on specific cloud accounts and clusters.

The dashboard consolidates critical security and operational data into the following widgets, allowing you to assess your security posture at a glance:

- Kubernetes Assets Overview: Obtain a high-level inventory of your Kubernetes environment, displaying the total count of clusters, namespaces, nodes, and workloads. Click on the drop-down icon for each asset to see a detailed breakdown of the Kubernetes platform type or workload. Click on an asset type to go to the Kubernetes Resources page, filtered according to that asset type.
- Cluster Protection Coverage: This widget shows the percentage of clusters that are fully protected for real time protection and posture management are installed on this cluster. protected with posture management, protected with real-time management, or unprotected. Identify your protection coverage gaps so that you can prioritize the riskiest clusters and onboarding of unprotected clusters. Click on the more options icon to see all clusters in the Kubernetes Connectivity Management page. Clicking on each state takes you to the Kubernetes Cluster page filtered by the state.
- Malware Detected: A list of cluster names and the total count of malware detected within them to allow for quick identification and response to active threats. Click on the more options icon to go to the Policy Management page.
- Cases and Issues: Displays the total number of cases and issues currently opened across your Kubernetes environment. Click on the more options icon to go to the Cases or Issues pages.
- Top Clusters by Vulnerabilities: Kubernetes clusters are ranked based on the number of high-severity vulnerabilities, ranked by CVSS score. Click on the more options icon to see all clusters in the Kubernetes Connectivity Management page or to go to the Vulnerability Policies Management page.
- Secrets Detected in Clusters: Shows the number of unsecured secrets found within your clusters. Click on the more options icon to go to the Policy Management page.

6 | KSPM Graph

The KSPM Graph provides a functional map of your infrastructure, moving away from static lists to an interactive visual interface. This allows for a direct drill-down workflow: navigate from global clusters down to specific workloads, nodes, and container images.

By mapping the actual visual map of your environment, the platform integrates security intelligence, such as vulnerabilities and misconfigurations, directly onto the relevant assets. This provides immediate context on how a specific security risk sits within your network.

Key capabilities:

- Risk prioritization: By presenting security posture as a hierarchical graph, security teams can easily identify which clusters, namespaces, and workloads carry the highest aggregated risk. Cortex then prioritizes aggregated risk based on all the findings, vulnerabilities, and internet exposure.
- Granular filtering: Apply client-side filtering with AND/OR logic to isolate specific resources that you want to focus on. namespaces, clusters, or labels. You can filter dimensions that include cloud provider, cluster name, cloud account, severity levels, and specific finding types. Filters persist seamlessly across your drill-down navigation.
- Managing limited visibility: A yellow status indicator on a cluster signifies limited visibility. This state occurs when the cluster connector is either non-functional or non-existent, typically due to a missing or incorrectly configured Kubernetes Posture Management (KSPM) connector. Without an active connector, the system cannot facilitate necessary data transmission.

  In the Asset card, from the More Options menu, you can select to Deploy Connector.
- Instant insights: The popovers on any node instantly display a detailed security breakdown. From there, navigate to the full asset detail view for investigation.

Evaluate your security posture

KSPM Graph organizes your Kubernetes infrastructure into three navigable levels, each providing progressively deeper visibility. Access is integrated directly from the Kubernetes Assets.

Level	Description
Cluster level	Displays all Kubernetes clusters across cloud providers in a grid layout. Each cluster node shows aggregated security metadata, including the total count of vulnerabilities, issues, malware findings, secrets, and internet-facing assets.
Namespace level	When you expand a cluster, the graph shows the namespaces and VM instances. To identify risk levels and compliance status, drill down into individual namespaces and workloads for detailed risk assessment and compliance information.
Workload and container image level	When you expand a namespace, the graph shows individual workloads, such as deployments and jobs, and runtime container images. This view shows security findings, vulnerability severity histograms, risk scores, and internet exposure status to help you investigate them directly.

6.1 | Asset detail card

To analyze a resource in more detail, select View Asset to open the asset card. The card includes these tabs:

Tab	Description
Overview	Provides a high-level summary of the resource's identity, security posture, and critical threats such as vulnerabilities or malware. It includes a health status of security scans and an interactive graph to visualize dependencies.
Resource Explorer	Provides a breakdown of associated assets, featuring visual graphs for Assets per Namespace and Assets per K8S Category, along with a detailed table listing resource specifics such as asset name, type, category, and tags.
Identity	Maps the permissions and access controls associated with the resource, illustrating relationships between its identity and associated roles or services.

Tab	Description
Configurations	Displays cloud configuration issues associated with the asset, tracking details such as severity and category. It also provides the raw Asset Configuration JSON for deep inspection of the asset's properties and metadata.
Vulnerabilities	Provides a detailed breakdown of all detected security findings associated with the resource. It offers a centralized view to help assess and prioritize the remediation of known security issues.
SBOM	Provides a comprehensive inventory of all software components and dependencies detected within the resource, tracking identification metadata and scan timing.
Network	Summarizes the resource's connectivity and exposure by displaying networking metadata and security controls. It allows users to analyze traffic boundaries, firewall policies, and network rules governing the assets across the cloud or cluster network.
Compliance	Evaluates the resource's adherence to regulatory requirements and industry benchmarks by presenting an overall security score. It provides a breakdown of audit results, highlighting which controls are met and which require immediate attention to mitigate risk.

NOTE:

Workloads context: The workloads section specifically identifies managed workloads, including Deployments, DaemonSets, StatefulSets, CronJobs, and ReplicaSets not managed by deployments, and jobs not managed by a CronJob.

6.2 | Security finding categories

KSPM Graph displays security finding categories on every node in the graph. Each category can be enabled or disabled independently.

Category	Description
Vulnerabilities	Displays a severity histogram showing the distribution of Critical, High, Medium, and Low vulnerabilities detected in your container images and workload configurations.
Issues	Highlights misconfigurations and policy violations detected by CIS Kubernetes Benchmark assessments and custom security policies.
Malware	Shows a count of malware detections identified through runtime analysis of container images and workload file systems.
Secrets	Highlights exposed secrets, such as API keys, credentials, and certificates, detected in container images or workload environment variables.
Internet exposure	Provides a Boolean indicator identifying workloads and container images that are reachable from the public internet.

7 | Kubernetes Clusters

The Kubernetes Clusters page provides a macro view of the entire cluster environment discovered by Cortex Cloud. The inventory view displays details about each cluster, including the cluster name, Kubernetes platform, Kubernetes version, cloud account, and whether the Kubernetes Connector is deployed. You can access the Kubernetes Clusters page by navigating to Modules → Kubernetes Security → Kubernetes Assets Inventory or from Inventory → Assets → All Assets → Compute.

The Kubernetes Clusters page provides a high-level overview of the containerized infrastructure through three primary data widgets:

- Kubernetes Clusters Distribution: Displays the total count of managed clusters segmented by their respective hosting platforms (e.g., EKS, GKE, OpenShift, AKS).
- Kubernetes Version Distribution: Provides a breakdown of the specific Kubernetes software versions currently running across the active environment.
- Protection Coverage: Offers a security status view that highlights the proportion of clusters where the security connector is either deployed or remains undeployed, indicating overall security visibility.

The Kubernetes Clusters table serves as the central location for granular cluster management. It offers a comprehensive, detailed view of each cluster, enabling administrators to assess the specific status and metadata of individual assets within the environment.

You can access the Kubernetes Connectivity Management page from the Kubernetes Clusters page to view cluster connectors and all related information.

Key functional aspects include:

- Asset visibility: Provides a centralized list of all clusters, including hosting platforms, versions, account identifiers, and custom tagging for easy categorization.
- Operational status: Allows you to track Last Seen and Last Scan timestamps to ensure real-time visibility and audit readiness across the environment.
- Connectivity & Remediation: Displays the current deployment state of security connectors. Click the Deploy button at the top of the page to launch the Kubernetes Connector and manage the deployment of security connectors for your clusters.
- Customization & Filtering: Supports flexible data manipulation through filter controls and configurable display settings to tailor the view based on specific investigation or reporting needs.

> **NOTE:**
>
> Data for the Resource Explorer and Vulnerabilities tabs are populated with data once the posture management solution is deployed.

Selecting any cluster in the table opens the detailed asset card with the following tabs:

- Overview: Summarizes the high-level identifying details and security posture of the cluster. Cluster details, including asset ID, provider, cloud region, and asset groups. If the Kubernetes Connector is deployed on this cluster, the connector details are also displayed. A clickable relationship graph allows you to visualize and understand the full context and dependencies of the cluster.
- Resource Explorer: Provides granular visibility into the specific assets deployed within the cluster. It includes a detailed list of discovered resources, including the namespace (when applicable), asset name, asset type, and category type.
- Identity: Displays the list of identities that can access this Kubernetes cluster.
- Configurations: Presents a list of security configuration issues detected within the Kubernetes cluster. For each issue, you can view the severity, name, category, and creation date. You can also view the asset configuration JSON.
- Vulnerabilities: Details all discovered vulnerabilities found within the container images running in the cluster, as well as for the host operating systems of your Kubernetes nodes.
- Compliance: Evaluates the resource's adherence to regulatory requirements and industry benchmarks by presenting an overall security score. It provides a breakdown of audit results, highlighting which controls are met and which require immediate attention to mitigate risk.

8 | Kubernetes Resources Inventory

The Kubernetes Resources page provides a detailed microview of the individual components deployed inside your connected Kubernetes clusters. The Kubernetes resources are tracked and managed within the broader Unified Asset Inventory (UAI). You can access the Kubernetes Resources page by navigating to Modules → Kubernetes Security → Kubernetes Assets Inventory → Kubernetes Resources or from Inventory → Assets → Kubernetes Resources.

The Kubernetes Resource page is a very granular inventory of Kubernetes objects discovered by the Kubernetes Connector:

- Workloads: Deployments, ReplicaSets, StatefulSets, DaemonSets, Jobs, and CronJobs
- Identity: ServiceAccounts, ClusterRoles, Roles, RoleBindings, and ClusterRoleBindings
- Network/Configuration: Services, Endpoints, Ingresses, NetworkPolicies, ConfigMaps, and Secrets

Use the filter at the top of the page to customize precisely which resources you want to view. Selecting any resource in the inventory opens the detailed asset card. The asset card provides a detailed overview, including asset properties, such as its ID, provider, region, and the cluster it belongs to. It also includes a clickable relationship graph to visualize and understand the full context and dependencies of the resource within the cluster. The asset card also includes the resource's code and vulnerability findings.

9 | Manage Kubernetes Connector instances

Abstract

You can manage the Kubernetes Connector instances on the Data Sources & Integrations page. You can check the status, edit, or delete Kubernetes Connector instances.

1. Navigate to Settings → Data Sources & Integrations.
2. Find the Kubernetes instance by clicking on the Kubernetes name or using the Search field.
3. In the row for the Kubernetes instance, click View Details. The Kubernetes Connectors page is displayed with all deployed Kubernetes Connectors. To view all Kubernetes clusters, including ones that are not yet deployed, go to the Kubernetes Connectivity Management page.
4. In the Kubernetes Connectors page, click on a cluster name to open the details pane for that instance.
5. You can perform the following actions on each Kubernetes Connector instance:

Action	Instructions
Open Cluster Details	In the details pane, click the more options icon and select Open Cluster Details. The Asset Card for that Kubernetes cluster is displayed.
Edit Connector	In the row for the Kubernetes instance, right-click and select Edit. Alternatively, in the details pane, click the more options icon and select Edit Connector. In Edit Kubernetes Connector, edit the configurations and click Apply Changes.You must execute the updated template in the Kubernetes environment for the configuration changes to be applied.
Delete Connector	In the row for the Kubernetes instance, right-click and select Delete. Alternatively, in the details pane, click the more options icon and select Delete Connector. To remove the connector, you must manually run Kubernetes commands to delete the resources in the Kubernetes environment. The commands are listed here.

Kubernetes Connectivity Management

Navigate to Settings → Data Sources & Integrations and find the Kubernetes instances by clicking on the Kubernetes name or using the Search field. In the Kubernetes Connectors page, click Kubernetes Connectivity Management to view all detected Kubernetes clusters. Here, you can check if a cluster is connected, view the status, and see the connector version. When a new version of the Kubernetes Connector is available, you can update it here.

> **NOTE:**
>
> After uninstalling the Kubernetes connector, the connector status updates to Not connected 48 hours after the uninstall process is initiated.

10 | Run an on-demand Kubernetes cluster scan

On-demand scanning applies to individual Kubernetes clusters managed by an active Cortex Cloud KSPM connector. On-demand scanning does not replace scheduled scan cycles and does not support bulk operations across multiple clusters simultaneously.

Prerequisites

Before requesting an on-demand scan, verify the following:

- A KSPM connector is deployed on the target cluster. If no connector is deployed, the context menu displays a Deploy connector option instead of Request scan.
- Ensure the Kubernetes connector for the target cluster is active and sends heartbeats to Cortex Cloud every 15 minutes.
- The KSPM connector is running version 2.0 or later. This version includes redefined cluster-level RBAC permissions that enhance security by restricting access to specific namespaces where possible.

> **NOTE:**
>
> If the Kubernetes connector for the target cluster sent a heartbeat more than 15 minutes ago, the Kubernetes Cluster Scan Now dialog displays the message: Action temporarily unavailable. The connector connectivity cannot currently be verified.

The Request scan option is available to all Cortex Cloud users with access to the Kubernetes Clusters inventory. No additional role-based permissions are required to request a scan.

Scan types

The Request scan dialog provides the following scan types:

Scan Type	Description	Cooldown Period	Default State
Inventory	Collects and updates the full inventory of Kubernetes resources in the cluster.	1 hour	Enabled
Nodes	Scans all nodes in the cluster, including container images, for vulnerabilities and misconfigurations.	6 hours	Disabled

> **IMPORTANT:**
>
> The nodes and containers scan is both CPU and memory-intensive. Run the scan no more than once every six hours to avoid performance degradation on the target cluster.

Each scan type enforces a cooldown period after a successful request. When a scan type is in cooldown, the corresponding checkbox is disabled, and a countdown badge indicates when the next scan request becomes available.

Request an on-demand scan

The Request scan option is available from two locations in the Cortex Cloud console:

From the Kubernetes cluster asset detail panel:

1. Go to Kubernetes Asset Inventory → Kubernetes Clusters.

2. Either right-click the target Kubernetes cluster row in the inventory table and select Request scan or open the asset detail panel and from the actions menu, select Request scan.
3. In the Request scan dialog, review the cluster details - cluster name, cluster distribution (EKS, AKS, GKE, OpenShift, or Kubernetes), cloud account name, and cloud provider.
4. Under Select scan type, select one or both scan types (Inventory scan, Nodes & containers scan) and then select Request.

> **NOTE:**
>
> If the connector version is earlier than 2.0, the Request scan option appears as Request scan (Update required) and is disabled. Upgrade the Kubernetes connector to version 2.0 or later to enable on-demand scanning.

Results

After a successful request, Cortex Cloud displays the Scan Requested confirmation, followed by one of the following messages:

- An Inventory scan was successfully requested
- A Nodes and Containers scan was successfully requested
- The connector is inactive (no heartbeat received in the last 15 minutes).

Known limitations:

- The scan executes after the next connector heartbeat (approximately 30 seconds), not immediately upon request.
- Bulk scan requests across multiple clusters simultaneously are not supported.
- Customization of cooldown periods is not available through the Cortex Cloud console.
- A historical audit log of on-demand scan requests is not available.

11 | Cloud Workload Policies and Rules

Cloud Workload Policies and Rules help organizations maintain security compliance, prevent misconfigurations, and reduce risks across cloud environments.

- Cloud Workload Policies define organizational security objectives by combining detection logic with preventive actions across selected asset scopes. Policies can generate issues and proactively block misconfigurations before they reach runtime, ensuring workloads remain compliant with security requirements throughout the Software Development Life Cycle (SDLC). They leverage identified security risks and enforce controls at the right stages of development and operations, such as during CI pipelines or in runtime environments.
- Cloud Workload Rules define the detection logic for misconfigurations and their applicable asset types, specifying the criteria and conditions used to identify security risks. These rules can be selected and enforced through Misconfiguration Policies within the designated policy asset scope.

Together, Policies define which risks must be addressed and what actions to take, while Rules specify how those risks are detected through precise logic and conditions.

> **PREREQUISITE:**
>
> Users need View/Edit RBAC permissions (under Policies → Compute Policies) or the Instance Administrator role to view, edit, and modify Cloud Workload Policies policies.

> **IMPORTANT:**
>
> Users with SBAC granular scoping (in addition to the RBAC permissions required for Cloud Workload Policies) can only view Cloud Workload Policies, when their access is scoped to any of the available options: All assets, No assets, or Select asset groups. For more information on granular scoping, see Manage user scope. When no SBAC restriction is applied, the user's access is determined solely by their RBAC permissions.

11.1 | How policies and rules work together

Cloud Workload Policies serve as enforcement mechanisms that govern the responses to the identified findings, whereas *Cloud Workload Rules* establish the criteria for evaluation but do not initiate any actions unless incorporated within a policy.

In the absence of an associated policy, rules exist solely as evaluative criteria and do not generate alerts or trigger any response actions. Policies determine how findings from rules are escalated to issues or preventive measures.

11.2 | Cloud Workload Policies

Cloud Workload Policies help you prevent and manage security violations in your cloud runtime instances. They enable you to apply detection logic to specific asset groups at the desired SDLC stage, and define what action needs to be taken if the conditions are met.

Depending on the nature of the security violation, a Cloud Workload Policy allows you to

- *Prevent the violation*. Enable proactive prevention of the violation. For example: Block an S3 bucket deployment that is open to the public.
- *Create an issue*. Create an issue when violation is seen. For example: Create an issue when an AWS credential file is found on a Linux server.

> **NOTE:**
>
> Issues are automatically resolved when the finding is no longer applicable to the asset or when the affected asset is removed from the inventory.
>
> For more details on Prevent and create issues, see Cloud Workload Preventive Action.

A Cortex Cloud Workload Policy has the following elements:

- SDLC Evaluation Stage: The SDLC stage at which the policy is applied and evaluated. Depending on the policy type, one or more of the following stages may be available:
  - CI: The stage during which a pipeline builds the artifact. After building the artifact, the pipeline pushes it to a registry.
  - Deploy: The stage when the artifact is pushed to a cloud instance for running.
  - Runtime: The stage when the artifact is running on a cloud instance.
- Rule (Conditions): The logical conditions that will trigger the evaluation of this policy.
- Scope: A filter specifying which assets the rule applies to.
- Action: The response triggered when the rule evaluates successfully (only when part of a policy). Based on the rules included in the policy, it can create an issue or prevent the security violation.

11.2.1 | Types of Cloud Workload Policies

- Misconfiguration policies: Enables you to assess various workloads for misconfigurations against relevant security standards and your organization's security guidelines. You can include both predefined and custom rules in these policies to either prevent violations or create issues for violations.
- Malware policies: Enable you to detect and manage malicious files within cloud workloads. These policies analyze files based on predefined parameters such as file name, path, size, and detection method.
- Secret policies: Enable you to identify and protect sensitive information—such as API keys and credentials—within workloads.
- Trusted Image policies: Enable you to ensure the authenticity, integrity, and security of container images and VMs deployed into your Kubernetes environments. This includes actions such as limiting allowed image sources, mitigating possible image tampering, and more.

11.2.1.1 | Trusted image cloud workload policies

Abstract

Trusted image policies ensure the integrity and security of container images and VMs deployed into Kubernetes environments. Using these policies, you can be assured that your images are from a trusted source, are built on approved and validated base images, and are free from unauthorized modifications.

Trusted image policies ensure the integrity and security of container images and VMs deployed into Kubernetes environments. This topic details the policy enforcement logic that Cortex uses to determine if an image is trusted, what action to take, and which issues to generate.

Images are evaluated to see if they:

- Match the trusted image policy criteria.
- Should be allowed or prevented, based on policy criteria and scope.
- Should trigger the creation of security or posture issues when untrusted.

> **NOTE:**
>
> Registry scanning is for finding problems that are objectively issues regardless of context, organization, or scope. Trust, however, is subjective. Depending on the scope and other factors, an issue may or may not be problematic, and can change over time depending on the context.

Trusted image policy actions

When defining a trusted image policy, you determine what actions should be taken if an image is not trusted.

- Create an Issue. The image is allowed to run, but a violation is recorded as a posture and/or security issue. For the purposes of this documentation, we refer to this as an **issue-only** action.

  The primary purpose of trusted image policies with the issue-only action is auditing and compliance. These violations are identified by Cortex periodic scans.

  The issue-only action assesses trust across a range of assets, including Kubernetes workloads (which are cloud-agnostic, supporting AWS, Azure, GCP, OCI), virtual machines (VMs).

- Prevent and Create an Issue. Images created after this policy is enabled will be blocked from running if they don't meet the trust criteria, and a violation will also be recorded as a security issue. For images that were already running—and which don't meet the trust criteria—a posture issue is created.For the purposes of this documentation, we will refer to this as the **prevent** action.

  These issues block deployment in real-time.

  The prevent action is cloud-agnostic (supports AWS, Azure, GCP, OCI) and enforced at the Kubernetes cluster level via the admission controller.

Before you begin working with trusted image policies

Consider the following prerequisite:

- All trusted image policies are designed for runtime security enforcement. However, to ensure that policies with the prevent action can successfully block untrusted images, the policy's scope must be on a Kubernetes cluster where the admission controller is enabled.

Trusted image policy enforcement logic

The evaluation of trusted image policies is based on logic that determines the following critical outcomes: Which container images are trusted in a given scope and which violations result in the creation of security and posture issues.

- Utilizing an "OR-based" approach

  Cortex utilizes an "OR-based" approach, with no explicit priority or rule order when resolving policy conflicts.

  This enables Cortex to prioritize the most permissive result among conflicting policies with prevent actions, while enforcing the most restrictive result between policies with issue-only actions.

  - Prevent actions: Minimizing prevention, and allowing as many images as possible to be trusted, is desirable in order to avoid blocking workloads from running.
  - Issue creation: Minimizing the number of issues generated is desirable to avoid issue redundancy, unnecessary analysis, and issue handling.

  Understanding the approach and the logic behind it helps you create policies efficiently.

  Recommendations include:

  - When possible, the optimal way to define all trust criteria is to define the criteria in one policy.
  - If you are defining cluster-level trust criteria along with other options at the namespace/registry level, understand that because there are no priority/order options, you must plan your policies accordingly.

- Trusting and allowing images by scope

  Trusted image policies provide granular control by ensuring that policies only impact the specific environment (scope) for which they are defined. An image can be trusted in one scope and untrusted in another. For example, a third party image may be trusted in your Development environment but immediately untrusted in your Production environment.

  If you have not defined any trusted image policies for a specific scope, the default behavior is that all images within that scope are implicitly trusted However, once a policy is defined for a scope, any image not explicitly trusted by that policy is automatically untrusted.

  Policy enforcement is directly tied to the defined scope. When you define the Asset Group scope for a policy (for example, a specific cluster's Test namespace), the policy applies only to the resources within that exact definition. The more narrowly defined your scope, the more targeted the policy's enforcement will be, ensuring the enforcement does not affect unrelated resources.

  If multiple policies with prevent actions cover the same scope, the logic is additive (allow-wins). As long as at least one of those policies results in the image being trusted, the image will be allowed to run.

  > **NOTE:**
  >
  > Trusted image policies do not override other policies and their rules, such as malware policies, misconfiguration policies, and secrets policies.

- Generating issues

  The following points clarify the logic for generating runtime security issues and posture issues based on your defined policies:

- Runtime security issues:

  A runtime security issue is generated only if an image is untrusted by all policies. If at least one policy trusts the image, no runtime security issue is created. Additionally, each policy with a prevent action that identifies an issue will generate only one corresponding runtime security issue.

- Posture issues:

  For policies with the prevent action:

  - if an image is prevented from running, no posture issues are created by any policy within that scope.
  - Posture issues are created if images that are running violate any policy within the scope.

  For policies with the issue-only action, posture issues are created at the granular level of one issue per untrusted image per policy that fails the trust criteria. The posture Issues are created on the relevant asset type and are titled accordingly:

  - Kubernetes workload asset type: The issue title will be Untrusted image running in Kubernetes workload X.
  - Virtual machine asset type: The issue title will be Untrusted image running in Virtual Machine Y.

  For each issue regardless of asset type, the description will be either:

  - For untrusted images:

    [AssetType] [AssetName] is running the image {ImageName}. that does not comply with the trust criteria defined in the Trusted Images policy [PolicyName].

  - For cases when there's missing information:

    [AssetType] [AssetName] is running the image {ImageName}. Information is missing to determine compliance with the trust criteria defined in the Trusted Images policy [PolicyName].

  The issues will also include the image name, a link to the image asset page, the relevant Kubernetes hierarchy (cluster and namespace) for Kubernetes workloads only, and more.

- Policy evaluation: Handling unknown, missing or partial image data

  Situations may arise where insufficient information is available at the time of deployment to conclusively arrive at a trust verdict. This means Cortex cannot confirm an image's trusted or untrusted status. Examples of such situations where there is a lack of complete image metadata include:

  - Initial discovery or missing scan data: If an image is being evaluated for the first time, some vital information may be temporarily unavailable that would generally have been derived from a prior comprehensive scan, such as the image's Base Layer data or its CLI scan status.
  - Partial deployment metadata: The trust criteria specified in your trusted image policy may include image metadata that is not present in the deployment file. For example, a policy might mandate a specific tag, but the Kubernetes deployment resource uses only the image digest (SHA) and omits the tag.

  Cortex handles these situations differently based on the policy's action:

  - For policies with issue-only actions: Cortex creates a posture issue and a runtime security issue, flagging the image as potentially untrusted due to insufficient data. The image is allowed to deploy, but the issue prompts the user to investigate the data gap.
  - For policies with prevent actions: Cortex relies on the user-defined Action when trust verdict is unavailable value in the policy's action section.

  > **CAUTION:**
  >
  > The default behavior for the prevent action when trust criteria are unavailable is Create an Issue. Be aware that changing this default will mean an incomplete deployment file can block a workload.

- Policy decision logic for workloads

  Situations might arise where within the same workload, multiple images or multiple policies yield conflicting trust results. When these conflicts occur—such as one policy trusting an image while another prevents it, or a single deployment containing both trusted and untrusted images—Cortex relies on the following defined logic to determine the final deployment outcome and what issues, if any, are generated.

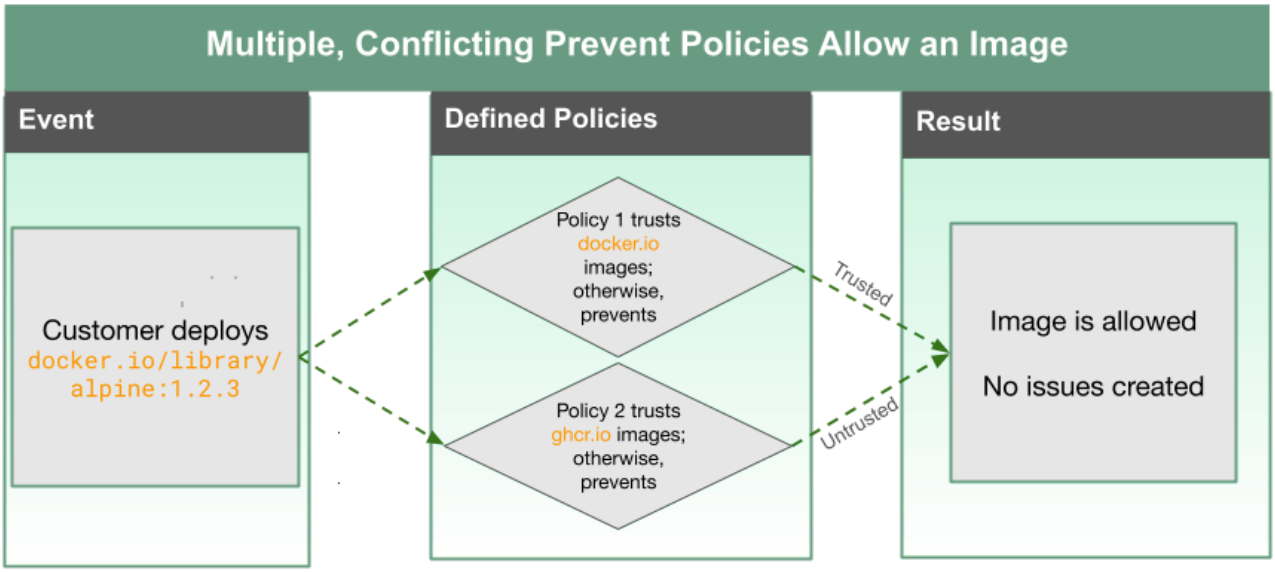
- For policies with issue-only actions: Any time an image fails the trust criteria for an Issue-only action policy, a Posture Issue is created. This action is non-blocking; the image is allowed to deploy regardless of the issue.
- For policies with prevent actions:
  - Conflicting allow vs. prevent policies: If multiple prevent action policies apply to a single image within a workload, and one policy allows the image while another does not trust it, the image is ultimately trusted and allowed to run. This follows the "Allow-Wins" principle.
  - Conflicting Images within the same workload: If a single workload (meaning, one deployment) contains both trusted and untrusted images, the entire deployment is blocked. The security risk posed by the single untrusted image prevents the entire workload from running.

Policy enforcement examples

This section provides examples that help illustrate the policy enforcement logic described above.

Example: Handling contradictory policies

This example illustrates how Cortex determines how to allow or prevent images, and create issues, when multiple policies conflict.

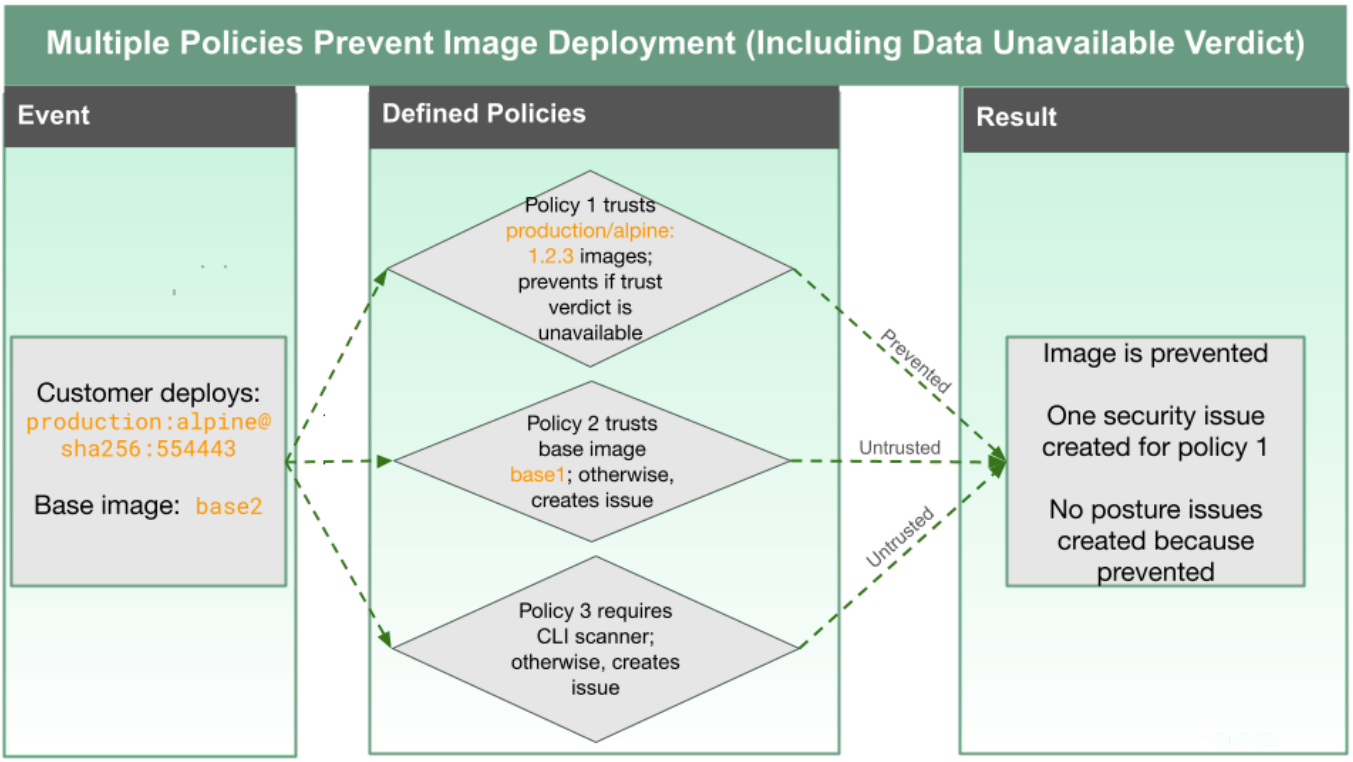


1. User deploys the docker.io/library/alpine:1.2.3 image.
2. Trusted image policies are validated at set periodic scan intervals or when a resource is deployed.
3. Two trusted image policies with prevent actions evaluate the image to determine its trust status. Cortex uses an "OR-based" evaluation for conflicting prevent action policies to determine if the image is trusted.
4. Policy 1 allows the image because the image is in the docker.io registry. So even though Policy 2 does not trust the image, the trust verdict is trusted and image deployment proceeds.
5. No runtime security issues are created because the image is trusted by one policy—namely, Policy 1.
6. No posture issues are created because:
   - There are no policies with issue-only actions—only policies with prevent actions.
   - The user defined two trusted registries, and a trusted image comes from one of them.

     The user does not anticipate any further issues.

Example: Preventing images based on policies with prevent actions

This example illustrates how Cortex determines how images are prevented from deployment in cases where a data verdict is unavailable.



1. User deploys the **production:alpine@sha256:554443** image with the base image of **base2**.
2. Trusted image policies are validated at set periodic scan intervals or when a resource is deployed.
3. Three policies with prevent actions evaluate the image to determine its trust status. Cortex uses an "OR-based" evaluation for conflicting policies to determine if the image is trusted.
    - Policy 1's criteria **production/alpine:1.2.3** only partially match the deployed image's full image identifier, **production:alpine@sha256:554443**. For example, the deployed image is missing the tag.

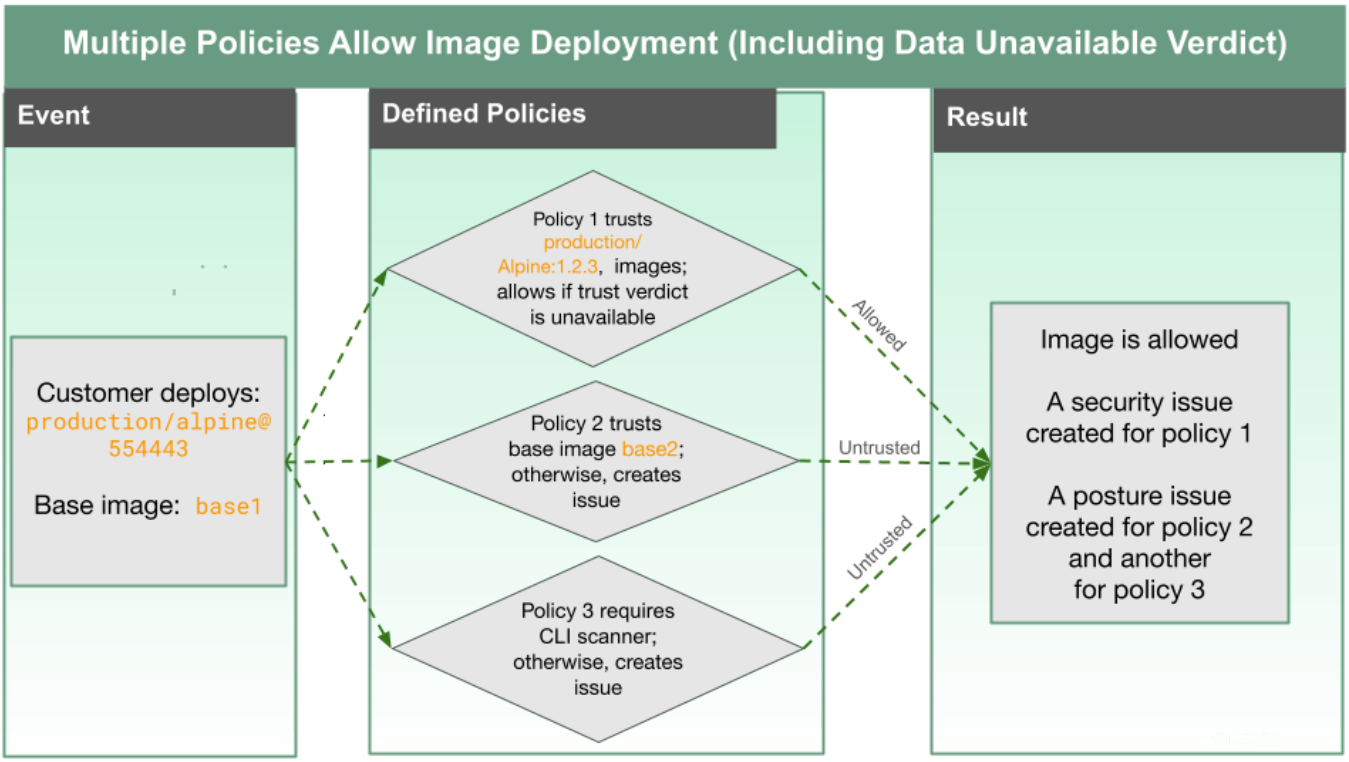
      If the image developer built the image, pushed it to the registry, and tagged it as **:1.2.3**, and that specific content generated the digest **sha256:554443...**, then they are a match at that moment in time. But this could change, so there is no definitive trust verdict.

      Because policy 1 has a prevent action with the additional **Action when trust verdict is unavailable** option set to **Prevent and create an issue**, the image is untrusted.
    - Policy 2, with its issue-only action, only trusts **base1** images, so this policy considers the image untrusted.
    - Policy 3, with its issue-only action, requires the image to have been scanned by a CLI scanner, so the image is untrusted.
4. Because all three policies do not trust the image, the image is prevented and deployment does not proceed.
5. A runtime security issue is created because of at least one policy does not trust the image. Only one runtime security issue is created, even though three policies don't trust the image.

   No posture issues are created because the image was prevented. Posture images must be actionable, and no actions can be taken on a prevented image.

Example: Allowing images based on policies with prevent actions

This example illustrates how Cortex determines how images are allowed to deploy in cases where a data verdict is unavailable.



1. User deploys the **production:alpine@sha256:554443** image with the base image of **base2**.
2. Trusted image policies are validated at set periodic scan intervals or when a resource is deployed.
3. Three policies evaluate the image to determine its trust status. Cortex uses an "OR-based" evaluation for conflicting policies to determine if the image is trusted.
    - Policy 1's criteria **production/alpine:1.2.3** only partially match the deployed image's full image identifier, **production:alpine@sha256:554443**. For example, the image is missing the tag.

      If the image developer built the image, pushed it to the registry, and tagged it as **:1.2.3**, and that specific content generated the digest **sha256:554443...**, then they are a match at that moment in time. But this could change, so there is no definitive trust verdict.

      Because policy 1 has a prevent action with the additional Action when trust verdict is unavailable option set to Create an issue, the image is trusted.
    - Policy 2, with its issue-only action, only trusts **base1** images, so the policy considers the image is untrusted.
    - Policy 3, with its issue-only action, requires the image to have been scanned by a CLI scanner, so the image is considered untrusted.
4. The image is allowed and deployment proceeds, because one of the three policies trusts the image and Cortex uses an "OR-based" approach.
5. No runtime security issue is created because of the trust verdict is to trust and allow.

   No posture issues are created because the image was prevented. Posture images must be actionable, and no actions can be taken on a prevented image.

11.2.2 | Cloud Workload Policies page

The **Cloud Workload Policies** page allows users to manage policies that define security and compliance actions for cloud workloads. Users can create, edit, filter, and manage policies through a structured table and widget panel.

> **NOTE:**
>
> Keep the following caveats in my mind when working with Policies:

- Instance Administrators are able to view all facets of policies without restrictions, even if Scope Based Access Control (SBAC) roles are in effect. Learn more about SBAC.
- If you've been assigned a custom role with View/Edit permissions limited by SBAC, you may not be able to view certain policies.
- You can further narrow your search on the Inventory page by using SBAC to limit the scope of the finding, issue, and case counts.

The Cloud Workload Policies page displays all the configured policies with the following fields.

Policy table columns

Field	Description
Policy Type	Defines the policy category: Misconfigurations, Secrets, Malware, Trusted Images.
Policy Name	The user-defined name of the policy.
Action	Defines the action taken when conditions match: Create an Issue (logs an issue) or Prevent and Create an Issue (prevents the action and logs an issue).
Severity	The severity level of the issue created: Critical, High, Medium, Low, or Informational.
Asset Groups	Predefined groups of assets to which the policy applies.
Open Issues	The number of unresolved issues associated with the policy.
Conditions	Define the detection rule by specifying the criteria that match relevant malware, secret, or trusted image findings.
Exceptions	Defines the exclusion criteria to omit malware, secret, or trusted image findings that meet specific conditions you want to exclude from the policy.
Evaluation Stage	Indicates at which stage in the SDLC the policy is evaluated.
Description	Additional details about the policy.
Created By	The user who created the policy.
Last Modified	The timestamp of the last modification.

11.2.2.1 | Widgets panel

The Cloud Workload Policies page includes a widget panel that provides a visual summary of policies:

- Policies by Type: Displays policies categorized as misconfiguration, secret, trusted images, or malware.
- Policies by Evaluation Stage: Shows the distribution of policies based on SDLC evaluation stages: Runtime, Deploy, or CI.

11.2.2.1.1 | Show or hide the widget panel

The widget panel provides a visual summary of policies based on policy type or evaluation stage.

To hide the widget panel, do the following:

1. Navigate to **Posture Management → Rules & Policies → Policies → Cloud Workload**.
2. On the Cloud Workload Policies page, click the Widget Panel icon at the top of the page.
3. The panel toggles between visible and hidden states.

11.2.2.2 | Change the layout of the policies table

1. Navigate to **Posture Management → Rules & Policies → Policies → Cloud Workload**.
2. In the Cloud Workload Policies page, click the More Options icon (⋮).
3. In the Layout tab, do the following:
    - To remove columns, go to the **In View** section and search for a specific column. Click **-** next to the column to remove it from the table.
    - To reorder columns, go to the **In View** section. Click and drag columns up or down to rearrange the columns.
    - To add new columns, go to the **Add Columns** section. Click **+** next to the columns to include them in the table.
4. The table layout updates automatically based on your selections.

11.2.2.3 | Policy Details Panel

The policy details panel is displayed when you click a policy in the policy table. To view details of a cloud workload policy:

1. Navigate to **Posture Management → Rules & Policies → Policies → Cloud Workload**.
2. In the **Cloud Workload Policies** page, select the policy you want to check.

The policy panel displays the following details related to the selected policy:

- Policy details.
- Related rule settings.
- The number of issues opened as part of the policy. You can click on the link to navigate to the Issues and Cases section to check the issue details.

From the policy detail panel, you can:

- Enable or disable the policy
- Edit the policy
- Save as new
- Delete the policy

11.2.3 | Enable or disable a Cloud Workload Policy

1. Navigate to **Posture Management → Rules & Policies → Policies → Cloud Workload**.
2. In the Cloud Workload Policies page, click on the policy you want to enable or disable.
3. In the Details page, click the toggle button at the top to enable or disable the policy.

11.2.4 | Create a Cloud Workload Policy

You can create policies to address specific types of security risks or compliance requirements.

To create a cloud workload policy:

Navigate to **Posture Management → Rules & Policies → Policies → Cloud Workload**.

In the Cloud Workload Policy page, click Create Policy and select the type of policy you want to create:

Misconfiguration Policy

1. Enter a unique name and description. Note that these are mandatory fields.
2. The Evaluation Stage will be selected as Runtime.

   > **NOTE:**
   >
   > The Evaluation stage for Misconfiguration policies is supported only in the Runtime SDLC stage and is enforceable through the Kubernetes Admission Controller for clusters on-boarded using the Posture Management (KSPM) Connector.

3. Click Next
4. The **Summary** section on the right displays a real-time, interactive view of all policy configurations as users progress through the wizard. It **automatically updates** to reflect the current selections and settings, enabling seamless navigation between fields from any step in the wizard. It includes the following sections:
   - **General** – Policy name and description.
   - **Rules** – No. of selected rules and the asset types relevant to the selected rules.
   - **Scope** - The defined scopes included in the policy and SDLC stage.
5. In the Rules section:
   - Click on Add Rules to select the rules that identify the violations that you want to track.

     A new window opens, displaying a list of all the existing predefined OOTB rules. See Rules filters for details on applying filters to refine the list of rules.

     > **NOTE:**
     >
     > Use Create a new Custom Detection Rule to define and add new custom rules as required.

   - After completing your selection, click Select to confirm. The chosen rules are displayed in the Rules section, where you can toggle between the Cards and Grid view to display the rules in your preferred layout.
   - In the Rules section, you can select one or more rules to modify the Severity, Policy Action and Remediation values, either individually or in bulk.

     > **NOTE:**
     >
     > Each rule may support different actions. While some include both Create an Issue and Prevent and Create an Issue, others provide only the Create an Issue option.

6. In the Scope section, for the Scope Selection Method , select Asset Groups or Default Asset Scopes, depending on your preference.

   1. If you select Asset Groups, you can choose between the following options:

   Select existing Asset groups

   The displayed list shows all Asset Groups available in the Runtime SDLC stageYou can select the asset group to which you want this policy to apply.

   Click Preview Selection to view all relevant Compute Assets associated with the selected asset groups.

   > **NOTE:**
   >
   > All non-relevant (non-compute) assets are automatically excluded from the included asset list.

   Add Group

   Click on Add Group. A new window opens to create a new **Compute Asset group**.

- Enter a unique Group name and Description.
- The displayed list of **Compute Assets** in the table is **pre-filtered** to show only the **relevant assets** based on the **rules selected in the previous section**. The **asset list is dynamically updated** and restricted to the applicable asset types of the selected rules, ensuring that users can select only valid and compatible assets for the new Compute Asset Group.

  You can filter these assets using the Show filter Panel button based on the fields Asset ID, Name, Provider and more.
- When only an **asset filter** is defined, the system creates a **dynamic asset group**, that automatically includes assets that meet the specified filter criteria.

  > **NOTE:**
  >
  > Currently, dynamic asset groups for Kubernetes Prevent Policies support only the following attributes:
  >
  > - **Kubernetes Resource Cluster**: The cloud identifier for the cluster.
  > - **Kubernetes Resource Namespace**: The namespace where the resource is located.
  > - **Kubernetes Resource Labels**: The labels assigned to the resource.
  > - **Kubernetes Resource Category**: The category of the resource (Identity, Workload, Configuration, or Network).
  > - **Kubernetes Resource Creation Time**: The time the resource was created.
  > - **Kubernetes Resource Name**: The name of the resource.

  When **specific assets are manually selected** from the asset list, a **static asset group is created**, containing only the explicitly chosen assets.

2. On selecting Default Asset Scopes, you can further select the Assets Scope from the predefined **Asset Scopes** that are filtered based on the **selected rules in the previous section** and their applicable **Asset Types**. The Scope options are dynamically updated and limited to the applicable asset types of the selected rules, ensuring that users can select only valid and compatible scopes.

7. Click Done to complete the process and create the new Misconfiguration Workload Policy.

Malware Policy

1. Enter a unique name and description. Note that these are mandatory fields.
2. Select an SDLC Evaluation Stage. The following options are available.
   - CI
   - Runtime
   - Deploy
3. Click Next.
4. The **Summary** section on the right provides a real-time, readable view of all policy configurations as users progress through the wizard. It automatically updates to reflect current selections and settings. It includes the following sections:
   - **General** – Policy name and description.
   - **Conditions** – Selected rule filter and exclusion criteria.
   - **Scope** - The defined scopes included in the policy and SDLC stage.
   - Actions - Selected action type.
5. Configure the settings specific to the evaluation stage you select.

   CI

- In the Conditions section, define the detection rule by specifying the criteria to identify relevant malware findings. You may also include exclusion criteria to filter out any malware findings that meet specific conditions you wish to exclude from this policy.
- Click Next.
- In the Scope section, select the checkbox to confirm that this selection applies the policy and its detection rules to All Cloud Workload Build Container Image asset types available at the CI SDLC stage.
- Click Next.
- In the Action section:
    - For Select an action, choose the option to Create an issue to log an issue if the policy is violated or Prevent and create an issue to prevent and create an issue.

      > **NOTE:**
      >
      > See Cloud Workload Preventive Action, to learn more about the Prevent action behavior and prerequisites.
      >
      > If the Prevent and create an issue action is selected, the preventive actions in the CI pipeline will trigger a Fail Pipeline by returning an exit code of 2 in the CI tool.

    - Under Issue Severity, choose Critical, High, Medium or Low to define the issue severity.
    - In the Remediation Guidance field, enter the optional remediation instructions.
- Click Done to complete the process and create the new Cloud Malware Workload Policy.

Runtime

- In the Conditions section, define the detection rule by specifying the criteria to identify relevant malware findings. You may also include exclusion criteria to filter out any malware findings that meet specific conditions you wish to exclude from this policy.
- Click Next.
- In the Scope section, for the Scope Selection Method, select Asset Groups or Default Asset Scopes, depending on your requirement.
    - If you select Asset Groups, you can choose between the following options:

      Select existing Asset groups

      The displayed list contains all the asset groups for the Cloud Workload Container Images, Container Instances, Hosts (VM Instances), Serverless Functions or Kubernetes Workloads asset types that are available at the Runtime SDLC stage.

      Add Group

        - Click on Add Group. A new window opens to create a new **Compute Asset group**.
        - The displayed list of **Compute Assets** in the table is **pre-filtered** to show only only the **Compute asset types** as Cloud Workload Container Images, Container Instances, Hosts (VM Instances), Serverless Functions or Kubernetes Workloads asset types, ensuring that users can select only valid and compatible assets for the new Compute Asset Group.

          You can filter these assets using the Show filter Panel button based on the fields Asset ID, Name, Provider and more.
        - When only an **asset filter** is defined, the system creates a **dynamic asset group**, that automatically includes assets that meet the specified filter criteria.

          When **specific assets are manually selected** from the asset list, a **static asset group is created**, containing only the explicitly chosen assets.

      You can then select the asset group to which you want this policy to apply.
    - On selecting Default Asset Scopes, you can further select the Asset Scopes as:
        - All Cloud Workload Assets)

          > **NOTE:**
          >
          > Choosing this option results in the automatic selection of all other asset scopes in the list.
        - All Cloud Workload Hosts(VM Instances)
        - All Cloud Workload Container Images
        - All Cloud Workload Container Instances
        - All Cloud Workload Kubernetes Workloads
        - All Cloud Workload Serverless Functions
- Click Next.
- In the Action section:
    - For Select an Action, choose the option to Create an issue to log an issue if the policy is violated or Prevent and create an issue to prevent and create an issue.

      > **NOTE:**
      >
      > See Cloud Workload Preventive Action, to learn more about the Prevent action behavior and prerequisites.
    - Under Issue Severity, choose **Critical, High, Medium or Low** to define the issue severity.
    - In the Remediation Guidance field, enter the optional remediation instructions.
- Click Done to complete the process and create the new Cloud Malware Workload Policy.

Deploy

- In the Conditions section, define the detection rule by specifying the criteria to identify relevant malware findings. You may also include exclusion criteria to filter out any malware findings that meet specific conditions you wish to exclude from this policy.
- Click Next.
- In the Scope section, for the Scope Selection Method, select Registry Images in Cloud Workload Asset Groups or the All Cloud Workload Registry Images, depending on your requirement.
    - If you select Registry Images in Cloud Workload Asset Groups ,this policy applies only to Cloud Workload Container Registry Images asset types in those groups that are available at the Deploy SDLC stage. A list of all these available asset groups is displayed. You can then select the asset group to which you want this policy to apply.
    - On selecting All Cloud Workload Registry Images, the policy and its detection rules will apply to All Cloud Workload Container Registry Images asset types available at Deploy SDLC Stage.
- Click Next.
- In the Action section:
    - For Select an Action, the default option is Create an issue to log an issue if the policy is violated.
    - Under Issue Severity, choose **Critical, High, Medium or Low** to define the issue severity.
    - In the Remediation Guidance field, enter the optional remediation instructions.
- Click Done to complete the process and create the new Cloud Malware Workload Policy.

Secret Policy

- Enter a unique name and description. Note that these are mandatory fields.
- Select an SDLC Evaluation Stage. The following options are available.
    - CI
    - Runtime
    - Deploy

- Click Next.
- The Summary section on the right provides a real-time, readable view of all policy configurations as users progress through the wizard. It automatically updates to reflect current selections and settings. It includes the following sections:
    - General – Policy name and description.
    - Conditions – Selected rule filter and exclusion criteria.
    - Scope - The defined scopes included in the policy and SDLC stage.
    - Actions - Selected action type.
- Configure the settings specific to the evaluation stage you select.

  CI

    - In the Conditions section, define the detection rule by specifying the criteria to identify relevant malware findings. You may also include exclusion criteria to filter out any malware findings that meet specific conditions you wish to exclude from this policy.
    - Click Next.
    - In the Scope section, select the checkbox to confirm that this selection applies the policy and its detection rules to All Cloud Workload Build Container Images asset types available at the CI SDLC stage.
    - Click Next.
    - In the Action section:
        - For Select an action, choose the option to Create an issue to log an issue if the policy is violated or Prevent and create an issue to prevent and create an issue.

          > **NOTE:**
          >
          > See Cloud Workload Preventive Action, to learn more about the Prevent action behavior and prerequisites.

        - Under Issue Severity, choose Critical, High, Medium or Low to define the issue severity.
        - In the Remediation Guidance field, enter the optional remediation instructions.
    - Click Done to complete the process and create the new Cloud Malware Workload Policy.

  Runtime

- In the Conditions section, define the detection rule by specifying the criteria to identify relevant malware findings. You may also include exclusion criteria to filter out any malware findings that meet specific conditions you wish to exclude from this policy.
- Click Next.
- In the Scope section, for the Scope Selection Method, select Asset Groups or Default Asset Scopes, depending on your requirement.
  - If you select Asset Groups, you can choose between the following options:

    Select existing Asset groups

    The displayed list contains all the asset groups for the **Cloud Workload Container Images, Container Instances, Hosts (VM Instances), Serverless Functions or Kubernetes Workloads** asset types that are available at the Runtime SDLC stage.

    Add Group

    - Click on Add Group. A new window opens to create a new Compute Asset group.
    - The displayed list contains all the asset groups for only the **Compute asset types** as **Cloud Workload Container Images, Container Instances, Hosts (VM Instances), Serverless Functions or Kubernetes Workloads**, ensuring that users can select only valid and compatible assets for the new Compute Asset Group.

      You can filter these assets using the Show filter Panel button based on the fields Asset ID, Name, Provider and more.
    - When only an asset filter is defined, the system creates a **dynamic asset group** that automatically includes assets that meet the specified filter criteria.

      When **specific assets are manually selected** from the asset list, **a static asset group is created**, containing only the explicitly chosen assets.

    You can then select the asset group to which you want this policy to apply.
  - On selecting Default Asset Scope, you can further select the Asset Scopes as
    - All Cloud Workload Assets

      > **NOTE:**
      >
      > **Choosing this option results in the automatic selection of all other asset scopes in the list.**
    - All Cloud Workload Hosts(VM Instances)
    - All Cloud Workload Container Images
    - All Cloud Workload Container Instances
    - All Cloud Workload Kubernetes Workloads
    - All Cloud Workload Serverless Functions
  - Click Next.
- In the Action section:
  - For Select an action, choose the option to Create an issue to log an issue if the policy is violated or Prevent and create an issue to prevent and create an issue.

    > **NOTE:**
    >
    > **See Cloud Workload Preventive Action, to learn more about the Prevent action behavior and prerequisites.**
  - Under Issue Severity, choose **Critical, High, Medium or Low** to define the issue severity.
  - In the Remediation Guidance field, enter the optional remediation instructions.
- Click Done to complete the process and create the new Cloud Malware Workload Policy.

Deploy

- In the Conditions section, define the detection rule by specifying the criteria to identify relevant malware findings. You may also include exclusion criteria to filter out any malware findings that meet specific conditions you wish to exclude from this policy.
- Click Next.
- In the Scope section, for the Scope Selection Method select Registry Images in Cloud Workload Asset Groups or the All Cloud Workload Registry Images, depending on your requirement.
  - If you select Registry Images in Cloud Workload Asset Groups ,this policy applies only to Cloud Workload Container Registry Images asset types in those groups that are available at the Deploy SDLC stage. A list of all these available asset groups is displayed. You can then select the asset group to which you want this policy to apply.
  - On selecting All Cloud Workload Registry Images, the policy and its detection rules will apply to All Cloud Workload Container Registry Images asset types available at Deploy SDLC Stage.
- Click Next.
- In the Action section:
  - For Select an Action, the default option is Create an issue to log an issue if the policy is violated.
  - Under Issue Severity, choose **Critical, High, Medium or Low** to define the issue severity.
  - In the Remediation Guidance field, enter the optional remediation instructions.
- Click Done to complete the process and create the new Cloud Malware Workload Policy.

Trusted Images

1. Enter a unique name and description. Note that these are mandatory fields. The SDLC Evaluation Stage is preset to Runtime.
2. Click Next.
3. Configure the policy's condition settings.

   a. In the Conditions section, specify the criteria to identify relevant images.

   You can specify criteria to define both broad policies and strict policies, for example: `Trust images (from registryX or registryY) OR (digestA or digestB)`. An example of criteria for a broad policy could be `all images from gcr.io/myorg/` while an example of criteria for a strict policy could be: `gcr.io/myorg/app@sha256:abc123`.

   You can also include exclusion criteria to filter out any images that meet specific conditions for exclusion from this policy.

   Because trust is subjective, context-dependent, and scope-based, we recommend you create finely-tuned criteria. For example, an image might be trusted in a low-risk demo environment because it has relaxed patching requirements, but it would be instantly blocked as untrusted in a production environment. For more information, see Trusted Image Policies.

   Considerations for specifying criteria for the policy's conditions

   - If you include a base image as a criterion in your trusted image policy, ensure that the base image itself is successfully scanned and pre-ingested into the system.
   - Do not use Scanned by CLI = Yes as the sole criterion for establishing image trust. The CLI scanning status is generally used as an indicator that contributes to overall trustworthiness. Instead, combine the CLI scanning status with stronger, verifiable identifiers like the image registry, signature, and/or base image.
   - Define trust criteria only using metadata that is consistently and explicitly included in your deployment files. The policy cannot establish trust if the required metadata (for example, a specific tag or label) is missing from the image definition.
   - Avoid using mutable tags as criteria for establishing image trust. Because the underlying image associated with a mutable tag can change without warning, basing trust on it can lead to unexpected outcomes. We strongly recommend enforcing immutable tags—a hallmark of secure, mature CI/CD pipelines—which is supported by all major registries (such as Docker Hub, ACR, ECR, GAR).
   - When using trust criteria that rely on image metadata, such as the base layer information or specific tags, we recommend that you pre-ingest the images into Cortex Cloud. While it is possible to base trust on an image which has not been pre-ingested, omitting this step can significantly impact the performance of your policy evaluation, which can slow down critical CI/CD workflows.

   b. Click Next.

4. Configure the policy's scope settings.

    a. In the Scope section, for the Scope Selection Method select Asset Groups or Default Asset Scopes.

    - Asset Groups. The policy applies only to Cloud workload container images, container instances, hosts (VM instances), serverless functions or Kubernetes workload asset types in those groups that are available at the Runtime SDLC stage. A list of available asset groups is displayed. You can then select the asset group to which you want this policy to apply.

      We recommend narrowing the asset group scope to ensure that a policy only checks relevant assets. For more information, see Trusted Images Policies.

      Consider the following when specifying criteria for the policy's scope:

      - Exclude system-critical Kubernetes namespaces, such as kube-system, from the policy scope to avoid interfering with core cluster operations.
      - If you select an asset group that contains a specific namespace, the policy will apply only to resources in that namespace—not the entire cluster.

    - Default Asset Scopes. On selecting Default Asset Scopes , you can further select the Asset Types:

      - All Cloud Workload Assets
      - All Cloud Workload Container Images
      - All Cloud Workload Kubernetes Workloads
      - All Cloud Workload Serverless Functions
      - All Cloud Workload Hosts (VM Instances)

    b. Click Next.

5. Configure the policy's action settings. In the Action section:

    a. For Select an Action, choose either Create an issue to log an issue if the policy is violated or Prevent and create an issue to prevent and create an issue. For more information, see Preventative action.

    b. If you select Prevent and create an issue as the policy's action, an additional Action when trust verdict is unavailable option becomes available. This is for situations where there is insufficient information available for determining if the image is trusted. The default is Prevent and create an issue.

    c. Under Issue Severity, choose Critical, High, Medium, or Low to define the issue severity.

    d. In the Remediation Guidance field, enter optional remediation instructions.

    Issues are automatically closed when the affected asset is removed from the inventory or when the policy is deleted. You can manually close issues at any time.

6. Click Done to complete the process and create the new policy.

11.2.5 | Use an existing policy to create a new Cloud Workload policy

1. Navigate to Posture Management → Rules & Policies → Policies → Cloud Workload.
2. In the Cloud Workload Policies page, click the policy you want to enable or disable.
3. In the Details page, click the More Options icon (⋮) and then select Save as new.
4. Modify the necessary fields in the Policy Name, Conditions, Scope, and Actions screens.
5. Click Done to create the new custom policy.

11.2.6 | Edit a Cloud Workload Policy

1. Navigate to Posture Management → Rules & Policies → Policies → Cloud Workload.

2. In the Cloud Workload Policies page, click the policy you want to enable or disable.
3. In the **Details** page, click the **Edit** icon.
4. Make the necessary changes.
5. Click **Done** to save the changes.

11.2.7 | Delete a Cloud Workload Policy

1. Navigate to **Posture Management → Rules & Policies → Policies → Cloud Workload**.
2. In the Cloud Workload Policies page, click the policy you want to delete.
3. In the **Details** page, click the **More Options** icon (⋮) and then select **Delete policy**.
4. Click **Delete** to confirm.

11.2.8 | Cloud Workload Preventive Action

Some Cloud Workload policies provide a Prevent and Create an Issue action that enforces compliance during deployments.

Prevention action for Runtime stage Policies

The Prevent action at Runtime applies only to Kubernetes Workload Images assets.

When a Kubernetes Workload image violates a policy, the Kubernetes Admission Controller (on clusters where the KSPM Connector is deployed and Admission Control is enabled) can block it from being admitted to the cluster.

For all other asset types within the policy scope, no runtime prevention will occur. Instead, the violation will result in an Issue being created.

Prerequisites

Ensure that your cluster has the Posture Management (KSPM) Connector deployed with the Admission Controller functionality enabled.

You can manage these deployments from the Kubernetes Connectivity Management page.

To access the Kubernetes Connectivity Management, navigate to the following URL in your tenant environment: https://[TENANT-ADDRESS]/cwp/k8s-management.

Important considerations

- **Recommended Approach**: Begin with the Create an Issue action to validate results before selecting Prevent and Create an Issue. This helps prevent potential disruptions to your applications or development workflows.
- **Impact on New Deployments**: The Prevent and Create an Issue action affects only new or future deployments that meet the prevention criteria. It does not impact cloud workload assets that are already deployed.

Prevention action for CI stage Policies

Prevention actions in the CI stage triggers a pipeline failure by returning an exit code of 2 in the CI tool.

11.3 | Cloud Workload Rules

Rules: Cloud Workload Rules define the criteria for identifying security violations. This criteria can be applied to assets in your cloud environment and to findings generated by Cortex Cloud.

Rules only enable the detection of security violations. They must be included in a policy to trigger a preventive response or generate an alert in the form of an issue.

11.3.1 | Default (pre-defined) Rules

Cortex Cloud includes a number of pre-defined rules to secure your cloud runtimes. These rules are used by default policies to prevent security violations and create issues.

11.3.2 | Custom (user-defined) Rules

Custom Rules or Custom Detection Rules allow you to define and implement tailored security and compliance checks within cloud workloads. These rules enable organizations to detect specific conditions, vulnerabilities, or misconfigurations that might not be covered by built-in system rules.

A custom rule consists of the following components:

- Scanner: Defines the mechanism by which the rule inspects the cloud assets. You need to select the scanner type that will implement the rule. Every time the selected scanner runs, all the rules associated with that scanner are also executed. The available scanner types are:
    - Agentless Disk Scan: Rules that use Agentless Disk Scanner to inspect the container images on which the Agentless scanner runs. You can specify different rules for containers running different OSes. For example, you can create a rule that checks for incorrect or malicious entries in the etc\hosts file on Windows images.
    - Kubernetes Connector: Rules that use the Kubernetes Connector scanner to inspect Kubernetes environment variables and resources such as Namespaces, ReplicaSets, Deployments and more.
    - XDR Agent: Rules that use XDR Agent Scanner to perform custom compliance checks by executing user-defined Python scripts, offering a tailored approach to compliance validation.
- Rule (Condition): Defines the detection criteria. This is specified as Rego or Python statements that evaluate assets, findings, and their associated attributes to identify security violations based on the selected scanner.
- Severity: The selected value is included in issues that are created as a result of rule violation.
- Compliance Controls: Associates the custom rule with a custom compliance control. If the rule detects the security violation, it will invoke the corresponding compliance control, thereby including the violation in relevant compliance reports.

11.3.3 | Cloud Workload Rules page

The Cloud Workload Rules page allows users to manage rules. Users can create, edit, filter, and manage rules.

> **NOTE:**
>
> Keep the following caveats in my mind when working with Rules:
>
> - Instance Administrators are able to view all facets of Rules without restrictions, even if Scope Based Access Control (SBAC) roles are in effect. Learn more about SBAC.
> - If you've been assigned a custom role with View/Edit permissions limited by SBAC, you may not be able to view specific Rules.
> - You can further narrow your search in a Rules table by using SBAC to limit the scope of the finding, issues, and case counts.

The Widget section enables the users to get 'at-a-glance' based on Platform, Rule type and Scanner type.

The Cloud Workload Rules page displays both the default rules and user-configured rules, with the following fields.

Rule table columns

Column Name	Description
Rule ID	A unique identifier assigned to each rule.
Rule Name	The name of the rule, typically defined by the user or system.

Column Name	Description
Description	A brief summary of the rule's purpose and functionality.
Policies	Lists the policies in which the rule is included.
Controls	Compliance controls associated with the rule for regulatory adherence.
Platform	Specifies the platform or environment the rule applies to. For example: Linux, Windows or Kubernetes.
Scanner	The tool or method used to evaluate findings, such as *Inventory Scanner*, *Agentless Disk Scan, Host Scanner, Kubernetes Connector or Kubernetes File System Scanner* .
Severity	Defines the severity of the rule.
Data Type	The type of data the rule evaluates. For example: Hosts or Kubernetes Resources
Created By	The user who created the rule.
Last Modified	The date and time the rule was last updated.
Rule Type	Indicates whether the rule is a Built-in or Custom rule.
Remediation	Defines the remediation steps to address the detected misconfiguration.
Applicable assets	Supported applicable asset types.
Available actions	Indicates whether the available action is Prevent and Create an Issue or Create an Issue
Standards	Associated compliance standards or controls
Open issue	No. of open issue related to this rule.

11.3.3.1 | Filter page results

You can use Show filter Panel button in the upper-right corner of the Rules page to filter the existing rules based on different filter criteria, as described below:

Table 1. Rule Filter table

Filter	Allowed Values
Rule Name	Rule names and empty values
Description	Rule description and empty values
Policies	No. of policies
Controls	No. of controls
Platform	*Linux, Windows and Kubernetes*
Scanner	*Agentless Disk Scan, Host Scanner, Kubernetes Connector, Kubernetes File System Scanner* and *Inventory Scanner*
Data type	*Hosts, Kubernetes Resources*
Severity	*Informational, Low, Medium, High* and *Critical*
Created by	System or specific username
Last modified	Selected date and time
Rule type	*Built-in* and *Custom*
Remediation	Remediation values
Applicable assets	Supported applicable asset types
Available actions	*Prevent and Create an Issue* and *Create an Issue*
Standards	Associated compliance standards or controls
Open Issues	No. of open issue
Rule ID	Unique Id of a rule

11.3.3.2 | Change the layout of the rules table

1. Navigate to **Posture Management > Rules > Cloud Workload**.
2. In the **Cloud Workload Rules** page, click the **More Options** icon (⋮).
3. In the **Layout** tab, do the following:
    - To add or remove columns, search for a specific column and:
        - Click **+** to add it to the table.
        - Click **-** to remove it from the table.
    - To reorder columns, go to the **In View** section and **click and drag** columns up or down.
    - To add new columns, go to the **Add Columns** section and click **+** to include them in the table.
4. The table layout updates automatically based on your selections.

11.3.3.3 | Rule details panel

The rule panel displays the following details related to the selected rule:

- Details of the rule like scanner details, remediation details and more.
- Compliance Controls for the rule.

This panel enables you to:

- Edit the rule
- Save as new
- Delete the rule

11.3.4 | Create a new Custom Detection Rule

Abstract

Create Custom Detection Rules to check your organization’s assets.

Creating Custom Detection Rules give you the flexibility to define and enforce security best practices tailored to your organization's objectives, as well as regulatory requirements not already covered by the compliance standards in our catalog.

Before you begin

Ensure you have a custom compliance control defined to associate the Custom Detection Rule to. For more information, see Use a built-in or custom standard.

How to create a Custom Detection Rule

1. Go to **Posture Management → Rules & Policies → Rules → Cloud Workload**.
2. In the Cloud Workload Rules page, click Create Custom Rule.
3. Enter the following settings:
    - Rule name: A descriptive name for the custom rule.
    - Description: An optional field for adding additional details or context about the rule, such as its purpose or intended behavior.
4. Select a Scanner to execute the Custom Detection Rule and its associated script. The options are:

- Agentless Disk Scan
- Kubernetes Connector
- XDR Agent

5. Configure settings specific to the scanner you select.

   Agentless Disk Scan settings

Field	Description
Operating System	The operating system targeted by the rule. The available options are: • Linux • Windows
Input file(s) path	The full file path for one or more files. For example, `/nfs/an/disks/jj/home/dir/file.txt`

Field	Description

Define the Rule (Rego)	Use Rego to define the custom detection logic.
	
	Use the default code in this box as a reference or starting point. Click read here for more information how to use Rego syntax.
	
	Example 1
	
	Code
	
	"/var/log/auth.log":
	{
	"content": "Failed password for invalid user test from 192.168.1.1 port 22 ssh2\n",
	"metadata":
	{
	"file_type": "file",
	"gid": 1000,
	"last_modified": 1737292449,
	"permissions": 436,
	"size": 6000,
	"uid": 1001
	},"path": "/var/log/auth.log"
	}
	
	Script
	
	package
	panw.complianceimport rego.v1
	
	match contains {"msg": msg}
	if {
	authLogFile = input["/var/log/auth.log"
	]
	contains(authLogFile.content, "Failed password")
	authLogFile.metadata.permissions == 436
	authLogFile.metadata.size > 5000
	msg := "Failed login attempts detected in /var/log/auth.log"}
	
	Output
	
	"match": [
	{
	"msg": "Failed login attempts detected in /var/log/auth.log"
	},
	]
	
	Example 2
	
	Code
	"/etc/passwd":
	{
	"content": "root:x:0:0:root:/root:/bin/bash\nuser1:*:1001:1001:User One:/home/user1:/bin/bash\n",
	"metadata": {
	"file_type": "file",
	"gid": 1001,
	"last_modified": 1737292449,
	"permissions": 644,
	"size": 100,
	"uid": 1002
	},"path": "/etc/passwd"
	}
	
	Script
	
	package
	panw.complianceimport rego.v1
	match contains {"msg": msg}
	if {
	passwdFile = input["/etc/passwd"]
	passwdFile.metadata.file_type == "file"
	passwdFile.metadata.permissions == 644
	passwdFile.metadata.size < 200
	contains(passwdFile.content, ":*:")
	msg := "Empty or suspicious password detected in /etc/passwd"}
	
	Output
	
	"match": [
	{
	"msg": "Empty or suspicious password detected in /etc/passwd"

Field	Description
	},] Example 3 Code "/etc/shadow": { "content": "root:6abc123$abc123abc123abc123abc123abc123abc123abc123abc123abc123abc123abc123abc123abc123abc123abc123:17542:0:99999:7:::", "metadata": { "file_type": "file", "gid": 1001, "last_modified": 1737292449, "permissions": 640, "size": 100, "uid": 1002 },"path": "/etc/shadow" } Script package panw.complianceimport rego.v1 match contains {"msg": msg} if { shadowFile = input["/etc/shadow"] shadowFile.metadata.file_type == "file" shadowFile.metadata.permissions != 600 shadowFile.metadata.size > 30 contains(shadowFile.content, "::") msg := "Empty or weak password detected in /etc/shadow"} Output "match": [{ "msg": "Empty or weak password detected in /etc/shadow" },]

Kubernetes Connector settings

Field	Description
Kubernetes Resources	From the drop down, select one or more from the following: • Namespaces: Logical partitions within a Kubernetes cluster that allow resource isolation and organization. • ReplicaSets: Ensures a specified number of pod replicas are running at all times by automatically scaling up or down. • Deployments: Manages and control pod replicas by providing declarative updates for ReplicaSets, enabling rolling updates and rollbacks. • StatefulSets: Deploys stateful applications that require persistent identity and storage, ensuring stable pod names and ordered scaling. • DaemonSets: Ensures that a copy of a specific pod runs on all or selected nodes in the cluster, commonly used for logging and monitoring agents. • Jobs: Runs one-time or short-lived workloads that complete execution and then terminate. • CronJobs: Defines scheduled jobs that run at specified times or intervals, similar to Linux cron jobs. • ClusterRoles: Defines permissions at the cluster level, granting access to resources across all namespaces. • Roles: Defines permissions within a specific namespace, restricting access to resources within that namespace. • RoleBindings: Associates a role with a user, group, or service account within a specific namespace. • ClusterRoleBindings: Associates a cluster role with users, groups, or service accounts at the cluster-wide level. • NetworkPolicies: Defines rules that control the communication between pods and other network entities within the cluster, enforcing security restrictions. • Services: Exposes a set of pods as a network service, allowing stable communication within and outside the cluster. • ServiceAccounts: Provides an identity for pods to authenticate against the Kubernetes API, allowing controlled access to resources. • Endpoints: Represents the actual network addresses of the pods backing a service, dynamically updated as pods start or stop. • Ingresses: Manages external access to services, providing HTTP/HTTPS routing, load balancing, and SSL termination. • ConfigMaps: Stores non-sensitive configuration data in key-value pairs, allowing applications to retrieve configuration without modifying container images. • Secrets: Securely stores and manage sensitive data, such as API keys, passwords, and certificates, in an encrypted format. • Nodes: Defines the physical or virtual machines that run the workloads in a Kubernetes cluster.

Field	Description

Define the Rule (Rego)	All custom Rego policies in Cortex must follow this pattern:
	`package panw.compliance`
	
	`import rego.v1`
	
	`match contains {"msg": msg} if {`
	`  # Your detection logic here`
	`  msg := "Description of the finding"`
	`}`
	
	NOTE:
	The custom rule must use the `match` term (not `deny` or others) to function properly.

XDR Agent settings

Field	Description
Custom Code Execution	Enable this setting for the scanner to perform custom compliance checks by executing user-defined Python scripts. **NOTE:** Only users with the following roles can enable or disable Custom Code Execution: • Account Admin • Instance Administrator • Deployment Admin • Privileged Security Admin Click Confirm to accept the following terms: • The Python scripts you provide will be executed in your cloud environment(s). • This capability is solely for the purpose of enabling you to define the compliance check rules for your cloud environment(s). Any other purposes are expressly prohibited. • Any actions involving WRITE, MODIFY, or DELETE operations of your cloud environment(s) are strictly prohibited. It is your responsibility to ensure that your custom Python scripts only perform read-only operations of your cloud environment(s) explicitly for compliance check purposes. • You are solely responsible for the quality, content, use, and execution results of your Python script. You assume all risks and liabilities arising from executing your Python script(s), including any potential errors, damages, or consequences resulting from its use. After you confirm accepting the terms, the rest of the XDR Agent settings appear.
Operating System	The operating system targeted by the rule. The available options are: • Linux • Windows

Field	Description
Define the Rule (Python)	Use Python to define the custom detection logic. This section supports syntax highlighting and validation (IntelliSense) to help users create accurate and efficient rules. Use the default code in this box as a reference or starting point. **IMPORTANT:** The custom Python scripts are intended to be executed exclusively for compliance checks and validations. To ensure the scripts are used properly and no security risks or unintended changes occur, the system implements the following restrictions and safeguards: • Only a predefined set of Python libraries and functions required for compliance checks are available for use. Libraries or functions that enable writing, deleting, or creating operations are excluded. • Only authorized users with specific permissions can create or update custom scripts. This ensures that only trusted individuals can define compliance checks.

6. For Compliance Violation Severity, define the severity level of the compliance violation to ensure proper categorization and prioritization. Possible values are:
   - Critical
   - High
   - Medium
   - Low
   - Informational
7. For Compliance Controls, assign the rule to one or more existing compliance controls.

   NOTE:

   Only Custom Detection Rules (not built-in rules) can be assigned to custom controls.

   a. Click Add.

   b. Select a custom compliance control from the list.

   c. Click Assign.
8. For Remediation, you can optionally define the remediation steps to address any detected misconfiguration.
9. Click Create.

   The new rule appears in the Rules List.

   You can now use the rule as a check to either create an issue or monitor adherence to a specific requirement.

   ##### Create an issue

   Under Posture Management → Policies → Cloud Workload, add the Custom Detection Rule to a Policy. This policy automatically runs the rule and creates an issue if the check fails.

   ##### Monitor compliance adherence

   Under Posture Management → Compliance → Catalogs → Standards, create a custom standard that includes the custom control associated with the Custom Detection Rule, and then create an assessment profile that runs the custom standard. You can then monitor the compliance results in a report. For more information, see Monitor and track compliance adherence.

11.3.5 | Use an existing rule to create a new Custom Detection Rule

1. Navigate to **Posture Management → Rules & Policies → Rules → Cloud Workload**.
2. In the **Cloud Workload Rules** page, click the policy you want to enable or disable.
3. In the **Details** page, click the **More Options** icon (⋮) and then select **Save as new**.
4. Modify the fields as required.
5. Click **Create** to create the new custom detection rule.

11.3.6 | Edit a Custom Detection Rule

1. Navigate to **Posture Management → Rules & Policies → Rules → Cloud Workload**.
2. In the **Cloud Workload Rules** page, click the rule you want to edit.
3. In the **Details** page, click the **More Options** icon (⋮) and then click **Edit**.
4. Make the necessary changes.
5. Click **Update** to save the changes.

11.3.7 | Delete a Custom Detection Rule

1. Navigate to **Posture Management → Rules & Policies → Rules → Cloud Workload**.
2. In the **Cloud Workload Rules** page, click the rule you want to delete.
3. In the **Details** page, click the **More Options** icon (⋮) and then select **Delete**.
4. In the confirmation message, click **Delete**.

12 | Create prevent policy for Kubernetes clusters

Learn how to create and apply custom cloud workload prevent policies for Kubernetes clusters using Cortex Cloud.

> **PREREQUISITE:**
>
> Ensure that the Kubernetes Connector is installed for this Kubernetes cluster and that admission controller enforcement is enabled.

1. Create a dynamic asset group for a Kubernetes cluster

An asset group is a collection of assets based on shared attributes. A dynamic asset group uses filters to group current and future assets that meet the defined criteria. Create a dynamic asset group that targets a specific Kubernetes cluster in Cortex Cloud.

1. Select Inventory → Assets → Groups → Add Group.
2. Define a meaningful Group Name that represents the group's purpose to improve usability.
3. Select the Kubernetes Resource Cluster filter and the operator Contains. Enter the name of your target Kubernetes cluster.
4. Click Create Dynamic Group to save and apply the asset group configuration.

2. Create a custom rule for the Kubernetes Connector

A cloud workload rule defines the criteria for identifying security violations. Custom rules allow you to define and implement tailored security and compliance checks within cloud workloads. These rules enable organizations to detect specific conditions, vulnerabilities, or misconfigurations that might not be covered by built-in system rules. Create a custom workload rule that is applied specifically to Kubernetes environment variables scanned by the Kubernetes Connector.

1. Select Posture Management → Rules & Policies → Rules → Cloud Workload → Create Custom Rule.
2. Define a meaningful Rule Name that represents the rule's purpose to improve usability. Optionally add a description of the rule to explain its purpose.

3. Under Scanner, select Kubernetes Connector.
4. Under Kubernetes Resources, select All to apply the rule across all supported resources.
5. Under Define The Rule (Rego), use Rego to define the custom detection logic. You can use the default code in this box as a reference or starting point. More information on how to use Rego syntax is available at the Open Policy Agent site.
6. Under Compliance Violation Severity, define the severity level of the compliance violation based on the impact or criticality of the rule.
7. Click Create.

   The new rule appears in the Rules List. You can now use the rule to create a custom compliance policy.

3. Create a custom cloud workload compliance policy

Cloud workload policies help you prevent and manage security violations in your cloud runtime instances. They enable you to apply detection logic to specific asset groups at the desired SDLC stage, and define what action needs to be taken if the conditions are met. Create a misconfiguration policy that includes the custom rule you created for the Kubernetes Connector and is applied to the asset group you created.

1. Select Posture Management → Rules & Policies → Policies → Cloud Workload.
2. In the Cloud Workload Policy page, click + Create Policy and select Misconfiguration policy type.
3. In the General page, define a meaningful Policy Name. Optionally add a description outlining the purpose of the policy. Click Next.
4. In the Conditions page, use the filter to find and select the custom rule you created. Click Next.
5. In the Scope page, select the asset groups that correspond to the Kubernetes clusters where the policy should be enforced. Click Next.
6. In the Action page, under Select an Action, select Prevent and Create an Issue.
7. Under Issue Severity, define the severity level according to your policy enforcement requirements.
8. Click Done to finalize and apply the new cloud workload compliance policy.

4. Verify Kubernetes admission controller policy enforcement

After the custom cloud workload policy has been applied, verify that it has been successfully pushed to your Kubernetes cluster and is functioning as expected.

1. Verify the cloud workload rule was deployed in Cortex Cloud

   Log into an instance that has access to your Kubernetes cluster and list the panrules deployed in the pan namespace:

   ```
   kubectl get panrules -n pan
   ```

   Output example:

   ```
   NAME                                     AGE
   e18734e2-a3b2-5e2d-a255-d73b302caee01    5m
   ```

   The rule name in the output should match the ID of the rule created above.

   > **NOTE:**
   >
   > If there are no rules listed, check that the cloud workload rule and policy were correctly configured and applied in Cortex Cloud.

2. Verify asset scope mapping

   Ensure that the asset scope is mapped to the intended Kubernetes cluster by checking the rule contents:

   ```
   kubectl get panrules 2d1fd464-9d94-4ba8-aa6d-6fcf576d0045 -n pan -o yaml | grep -i assetScope
   ```

   Output example:

   ```
   assetScope:
   eyJBTkQiOlt7IlNFQVJDSF9GSUVMRCI6InhkbS5rdWJlcm5ldGVzLnJlc291cmNlLmNsdXN0ZXIiLCJTRUFSQ0hfVFlQRSI6IkNPTlRBSU5TIiwiU0VBUkNIX1ZBTFVFIjoiZmFh
   aG1lZC1jb3J0ZXgtY2xvdWQtZGVtbyJ9XX0
   ```

   Decode the asset scope, which is base64-encoded:

   ```
   echo
   eyJBTkQiOlt7IlNFQVJDSF9GSUVMRCI6InhkbS5rdWJlcm5ldGVzLnJlc291cmNlLmNsdXN0ZXIiLCJTRUFSQ0hfVFlQRSI6IkNPTlRBSU5TIiwiU0VBUkNIX1ZBTFVFIjoiZmFh
   ```

```
aG1lZC1jb3J0ZXgtY2xvdWQtZGVtbyJ9XX0= | base64 -d
```

Output example:

```
{
  "AND": [
    {
      "SEARCH_FIELD": "xdm.kubernetes.resource.cluster",
      "SEARCH_TYPE": "CONTAINS",
      "SEARCH_VALUE": "jsmith-cortex-poc"
    }
  ]
}
```

Ensure that the "SEARCH_VALUE" matches the name of the cluster you are targeting.

3. Test policy enforcement

   Deploy a sample that violates the custom policy (for example, using an image without the `:latest` tag). This is a sample `nginx.yml` file:

   ```
   apiVersion: apps/v1
   kind: Deployment
   metadata:
     name: nginx-deployment
     labels:
       app: nginx
   spec:
     replicas: 3
     selector:
       matchLabels:
         app: nginx
     template:
       metadata:
         labels:
           app: nginx
       spec:
         containers:
         - name: nginx
           image: nginx:1.20
           ports:
           - containerPort: 80
   ```

   Apply the deployment:

   ```
   kubectl apply -f nginx.yml
   ```

   Expected error:

   ```
   Error from server: error when creating "nginx.yml": admission webhook "admission-control-validating-config.pan.svc" denied the request:
   image 'nginx:1.20' can’t deployed old image in Namespace default.
   ```

   This confirms that the admission controller is working correctly and is enforcing the custom policy.